



Large Language Models

Advanced Software-Engineering
Dr. Harald Stein
Prof. Dr. Selcan Ipek-Ugay
WS 2026/27



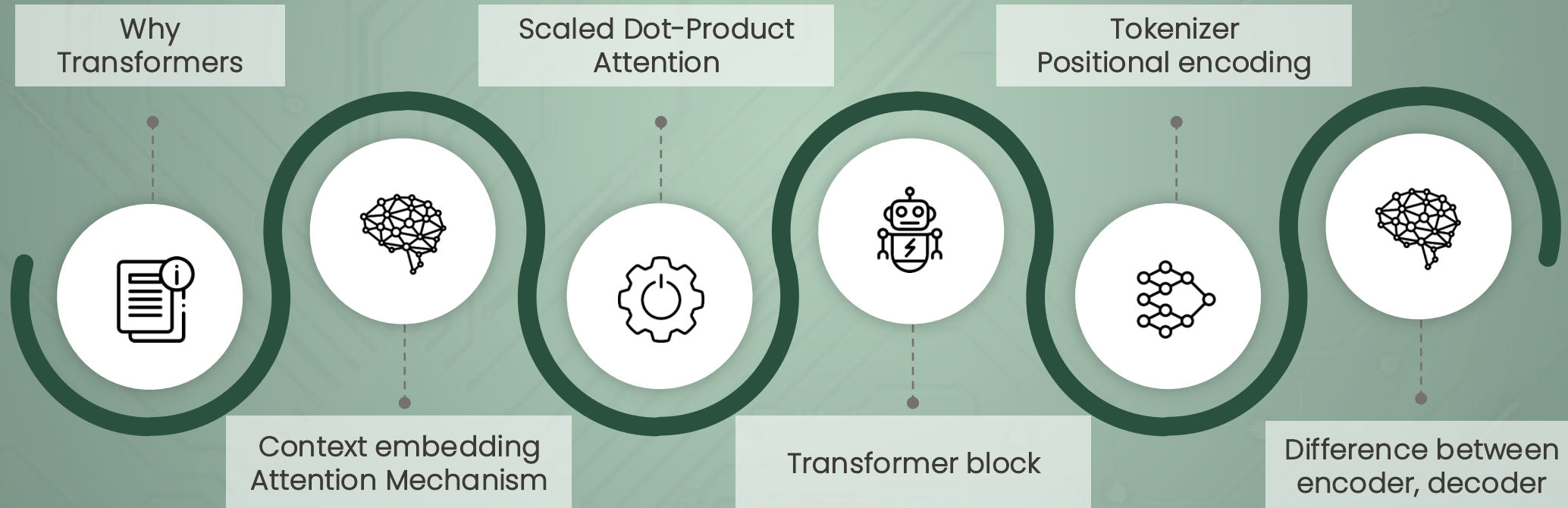
Agenda

- **Transformers based architecture**
- **Training Large Language Models**
- **Augmenting data from the outside**
 - Prompt Engineering
 - RAG
- **Evaluating LLMs**
- **Programming Tools**



Transformers

... are type of neural network architecture that uses self-attention mechanisms to process entire input sequences in parallel, enabling efficient handling of long-range dependencies



Why Transformers are significant

Transformers excel at modeling sequential data like natural language.
Nowadays encoders, decoders are used separately.

Encoder:

- Sentiment Analysis
- Text classification

Decoder:

- Conversation (ChatGPT)
- Translation

Comparison with RNNs

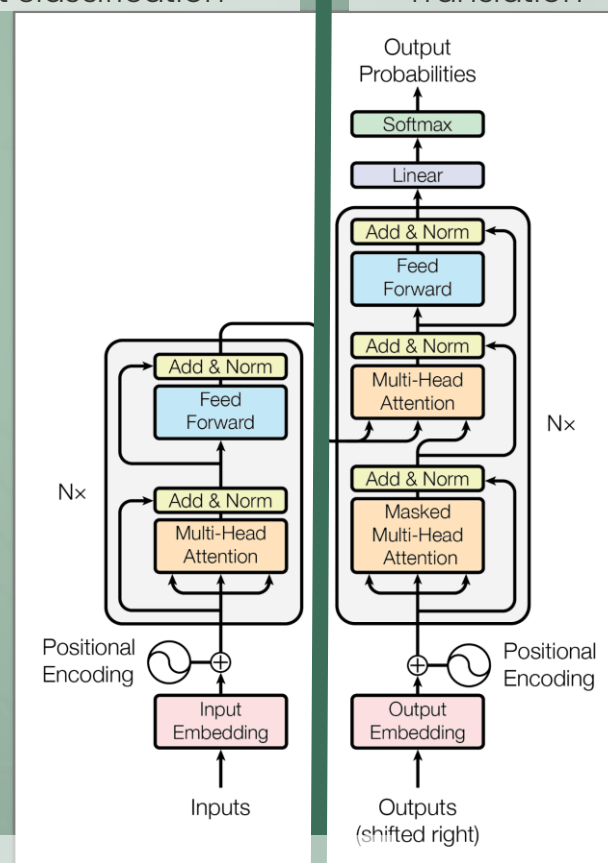
- Parallelizable and efficient on GPUs & TPUs.
- Replaces recurrence with attention for simultaneous computations.
- Outputs computed in parallel, unlike RNNs' series.

Advantages Over RNNs and CNNs

- Captures distant or long-range contexts in data.
- Connects distant positions in sequences for longer connections.
- Uses attention to access entire input at each layer, unlike RNNs, CNNs.

Unique Characteristics

- No assumptions on temporal/spatial relationships.
- Ideal for processing sets of objects (e.g., StarCraft units).



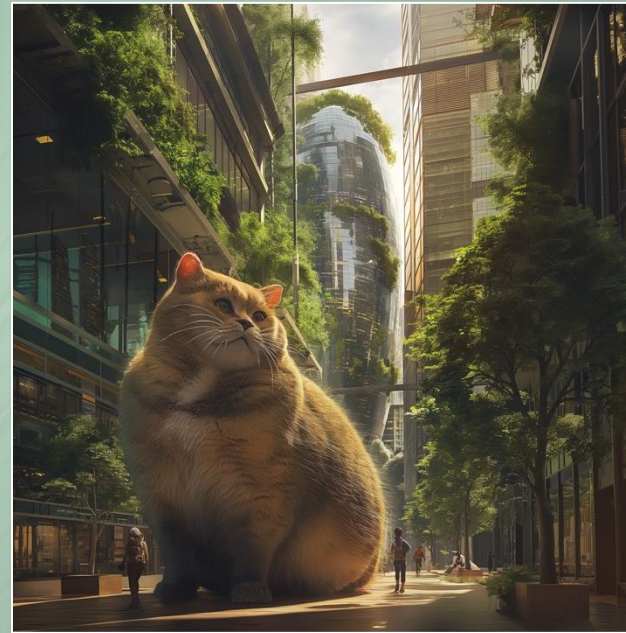
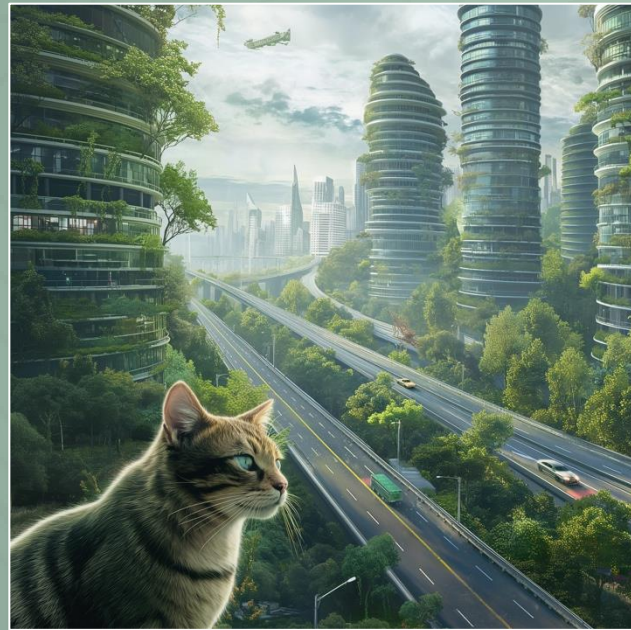
Paper:

Attention is all you need (2017)

Context embedding

The cat was afraid to pass the street because it was too wide.

Result



Does "it" refer to cat or to street?

Based on training humongous amount of text data the transformer learns that in this context "it" refers to "street"

→ "it" has maximal attention to "street"

Context embedding

The cat was afraid to pass the street because it was too tired.



Result

Does "it" refer to cat or to street?

Based on training humongous amount of text data the transformer learns that in this context "it" refers to "cat"

→ "it" has maximal attention to "cat"

Attention Mechanism

A technique in deep learning models, especially in sequence-to-sequence tasks, that allows the model to focus on specific parts of the input when producing an output.

Key Features

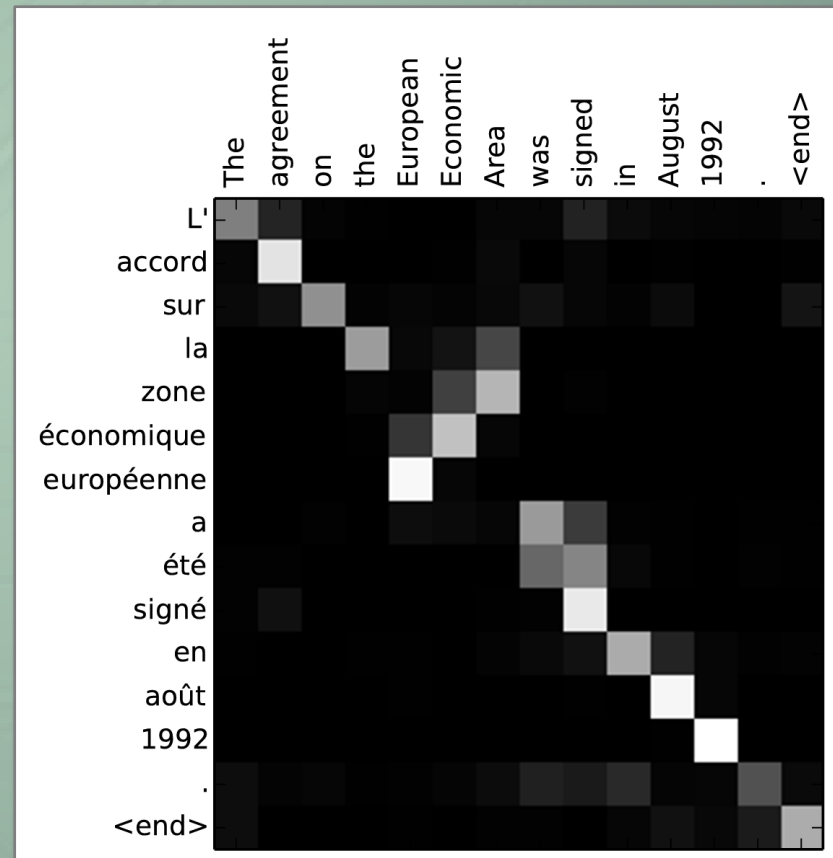
- Dynamically weighs input elements.
- Enhances the capturing of long-range dependencies in sequences.

Usage

- Machine Translation: Helps in aligning words in source and target languages.
- Text Summarization: Prioritizes crucial parts of the content.

Benefits

- Improves model's ability to remember long sequences.
- Enhances accuracy in tasks like translation and summarization.



Attention Mechanism

What is the most probable missing word? It is determined by putting the other words into context to each other.

"Sarah lies still on the bed feeling ____"



"Sarah **lies** still on the bed feeling ____"

The diagram illustrates an attention mechanism. It shows a sequence of words in a sentence: "Sarah", "lies", "still", "on", "the", "bed", "feeling", and a blank space. Above the words "feeling" and the blank space, there are several question marks. Arcs of varying thickness connect the word "feeling" to the question marks. The thickest arc connects "feeling" to the question mark immediately above the blank space, indicating that the model is most focused on this word to determine the missing word. Other thinner arcs connect "feeling" to other question marks, showing a distribution of attention across the context.

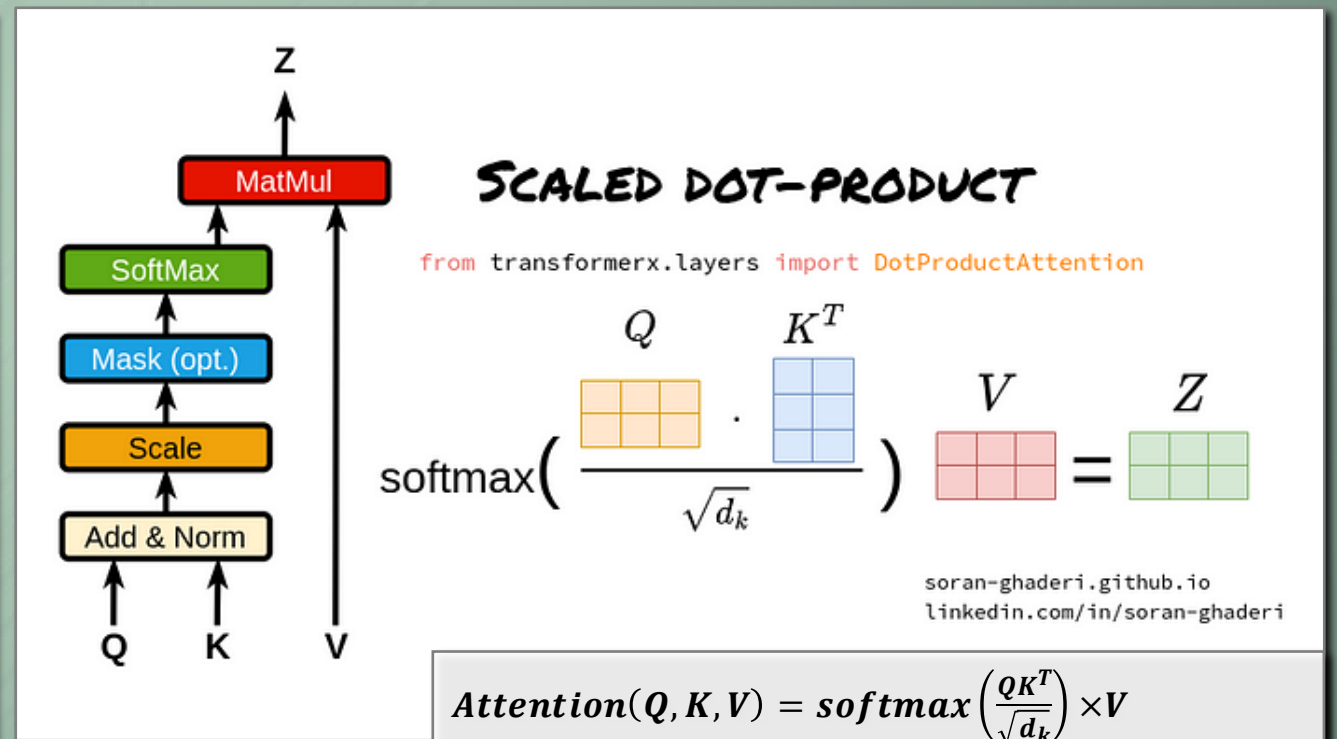
Scaled Dot-Product Attention

...is a mechanism used in attention models that calculates attention scores based on dot product of query and key, scaled down by square root of their dimensionality.

- Queries: derived from input data, represent focus of model
- Keys: also derived from input data, interpretation: "labels" for the input data
- Values: are weighted based on compatibility of query and corresponding key

How it works

- For each query, attention mechanism computes score with each key in input. Score represents how well query aligns with particular key.
- Scores are used to create a weighted combination of the values.
- If key aligns well with query, value associated with that key gets larger weight in final output.



$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right) \times V$$

- Q, K, V : Query, Key, and Value vectors respectively
- d_k : dimensionality of query/key vectors

Attention Mechanism: Value (V) vectors for all words

These contain actual information of each word that will be aggregated to form output

VALUE



Sarah lies still on the bed feeling

Position is not important for coding, if word occurs several times it receives the same code

[0.1, 0.2]

[0.3]

[0.4]

[0.5]

[0.6]

[0.7]

[0.8, 0.9]

Attention Mechanism: Key (K) vectors for all words

These represent how each word in the sentence can be queried



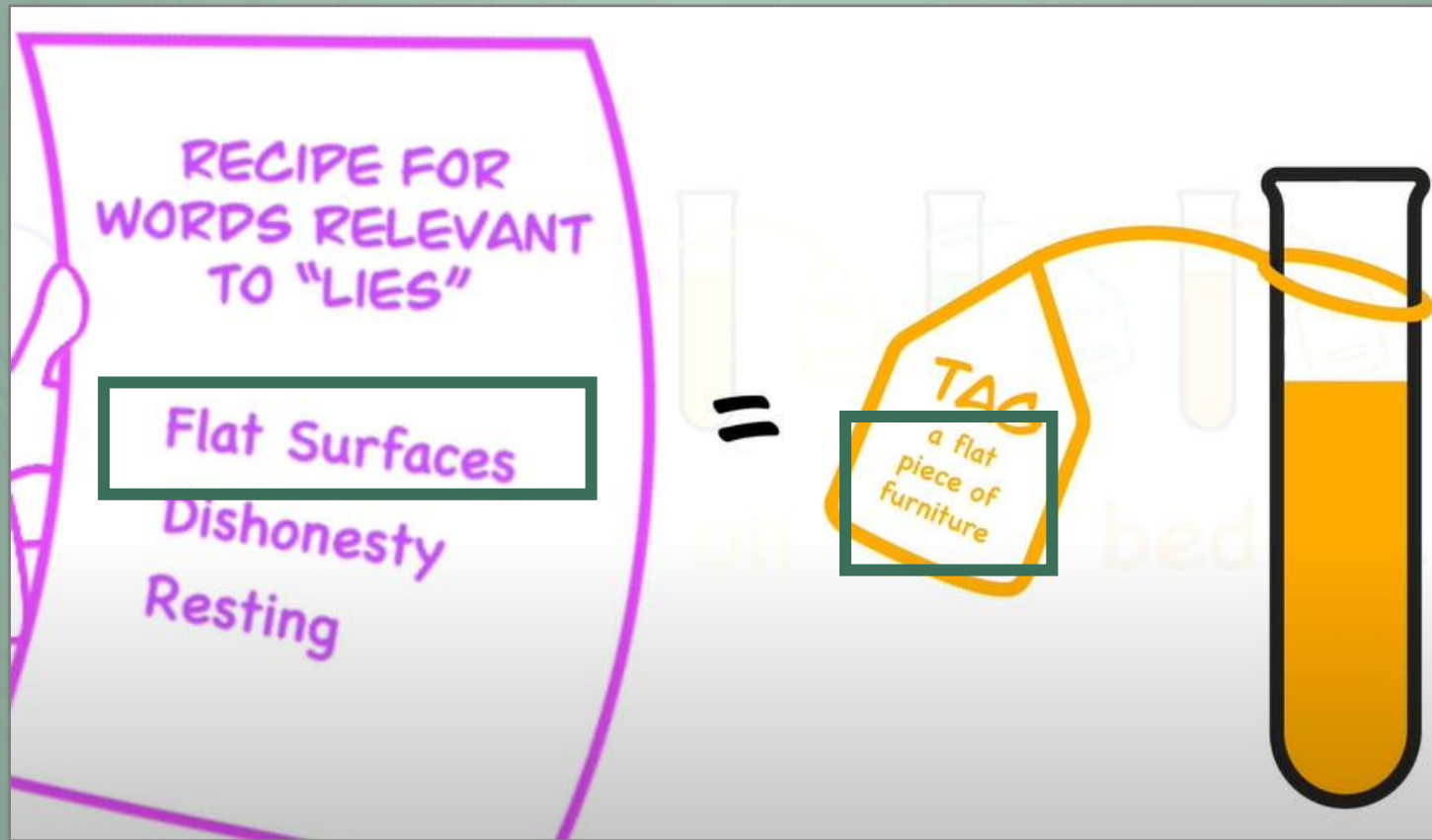
Attention Mechanism: Query (Q) vectors

... are used to represent the word in question, seeking information from other words.



Attention Mechanism: Connecting query and key

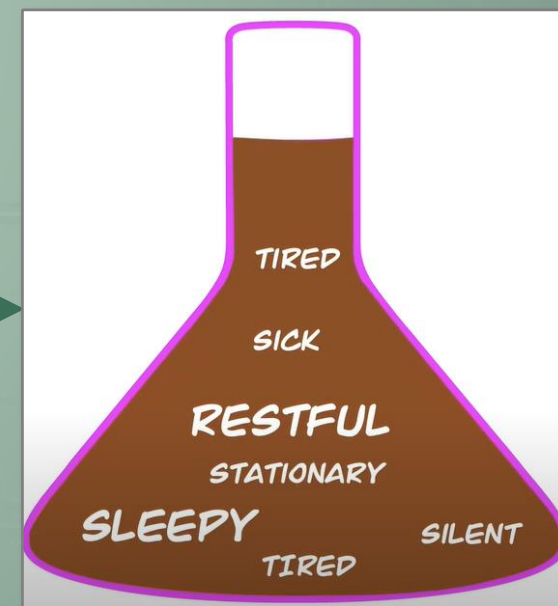
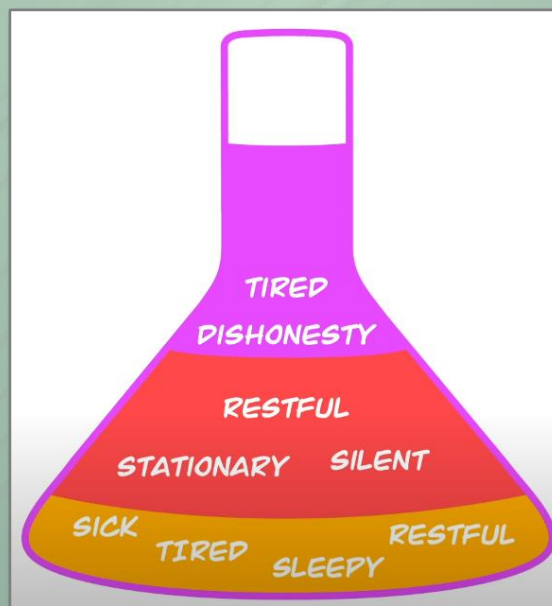
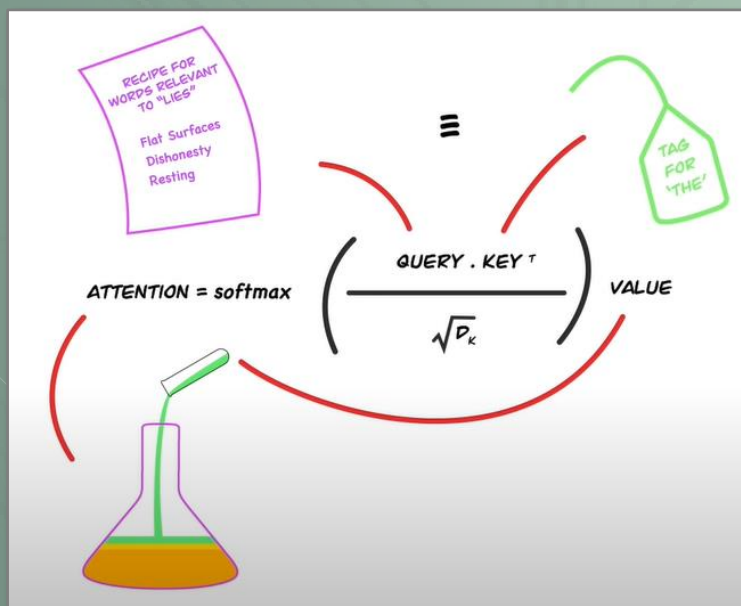
... "Flat Surfaces" from Recipe and "a flat piece of furniture" are similar



Attention

... of the word "lies"

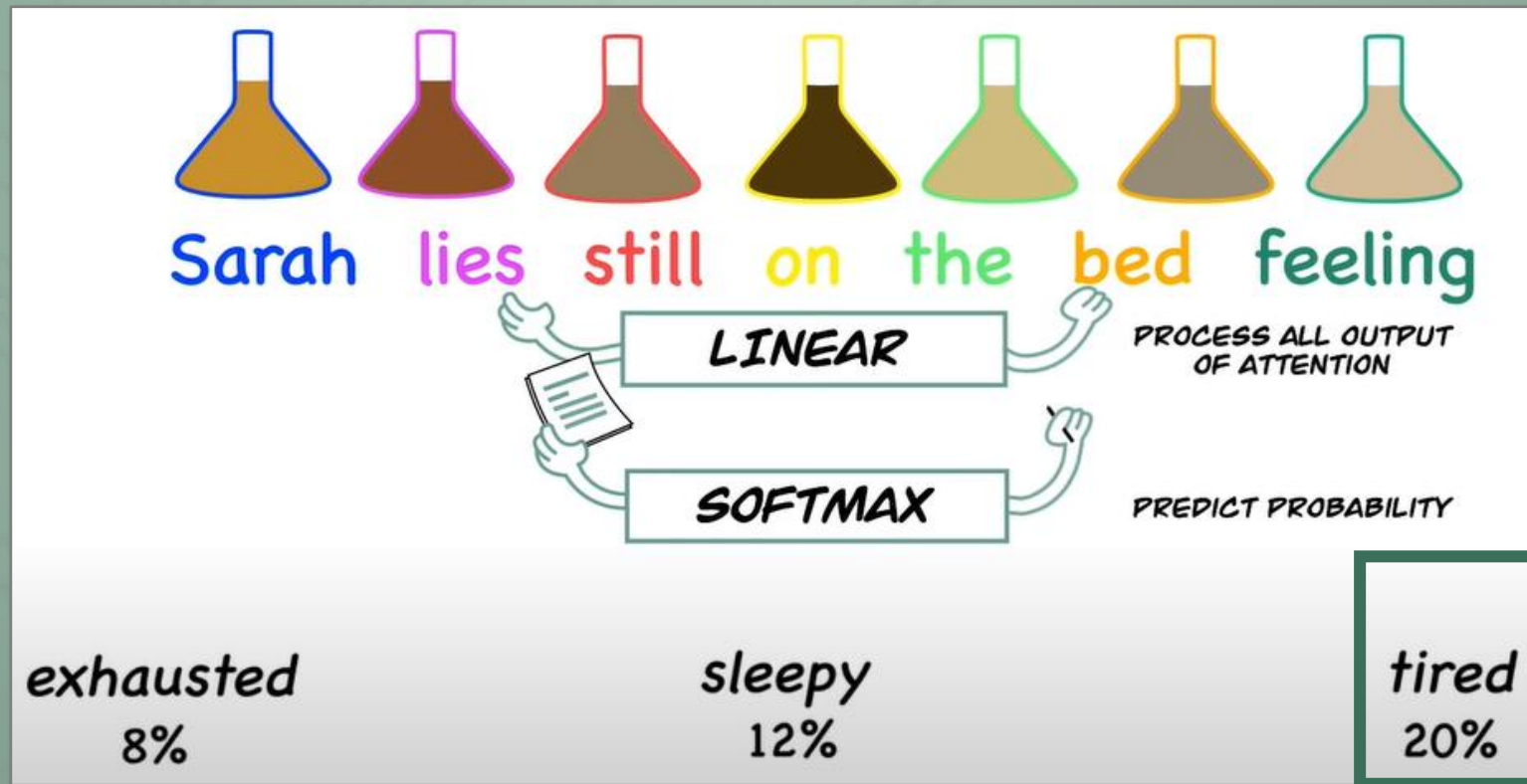
"Sarah **lies** still on the bed feeling _____"



Attention Mechanism

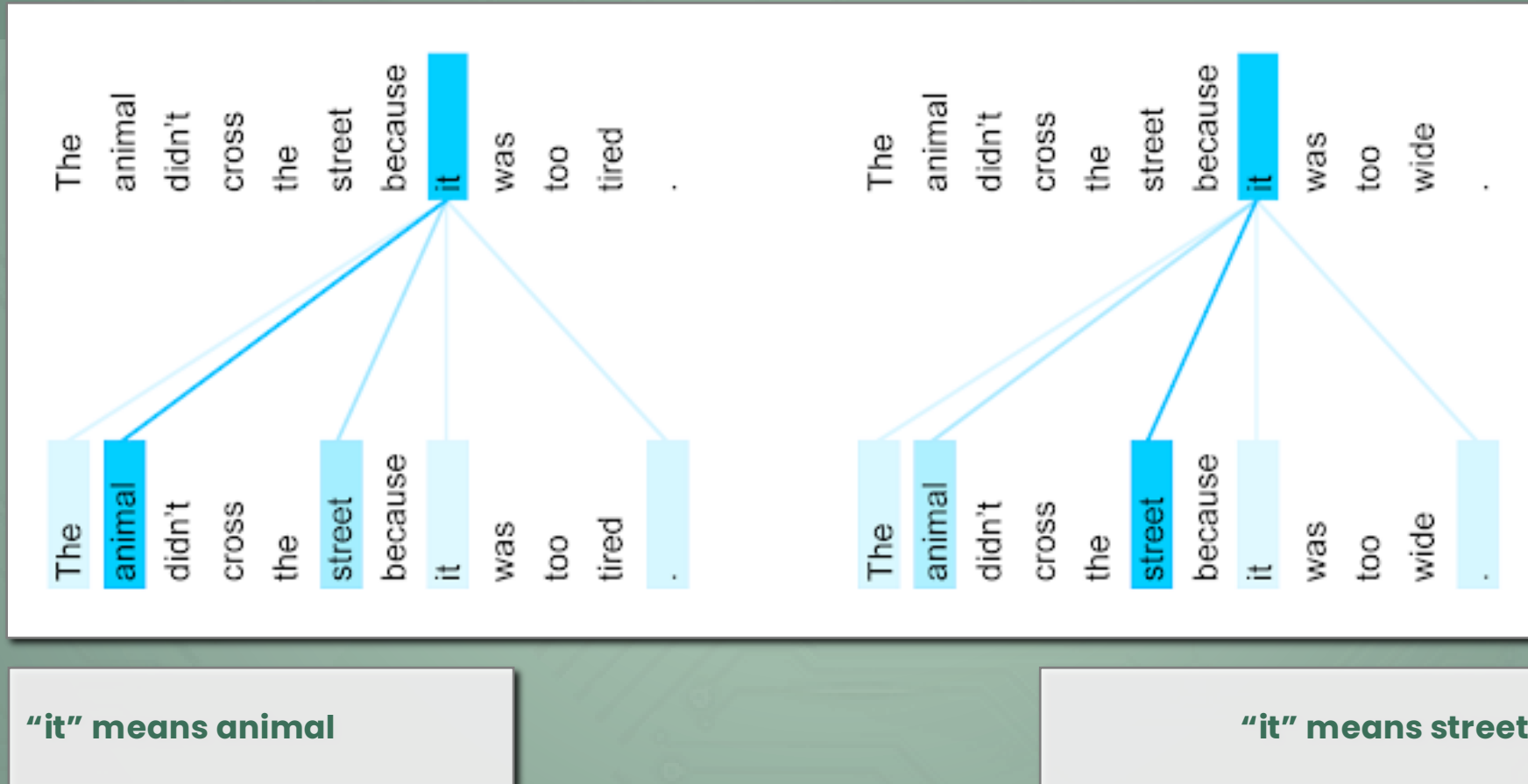
... calculates the word with highest probability through linear transformation & softmax

"Sarah lies still on the bed feeling_____"



Self-attention distribution: word “it”

Meaning of the word „it“ depends on context. Any complex task like translation or even Sentiment Analysis of multi-sentence comments needs context awareness.



Attention Mechanism

What is the most probable missing word?

It is determined by putting the other words into context to each other.

Colors

- Red: represents current word being fixated
- Blue: represents memories.
- Shading: indicates degree of memory activation

Prediction as result of Training

- Sentence was split as triangle, i.e. each word was represented by one row with all previous words
 - Red words: Predicted target values
 - All previous words: input values

→ Self-supervised learning

The FBI is chasing a criminal on the run .

The FBI is chasing a criminal on the run .

The FBI is chasing a criminal on the run .

The FBI is chasing a criminal on the run .

The FBI is chasing a criminal on the run .

The FBI is chasing a criminal on the run .

The FBI is chasing a criminal on the run .

The FBI is chasing a criminal on the run .

The FBI is chasing a criminal on the run .

The FBI is chasing a criminal on the run .

Long Short-Term Memory-Networks for Machine Reading (2016)

Multi-head Attention

... is key component of transformer models, allowing the model to focus on different parts of input sequence simultaneously, by applying multiple attention operations in parallel.

Multiple Attention Heads

- Uses sets of Q, K, V weight matrices
- Each set forms an 'attention head', attention scores are calculated for each head

Parallel Processing

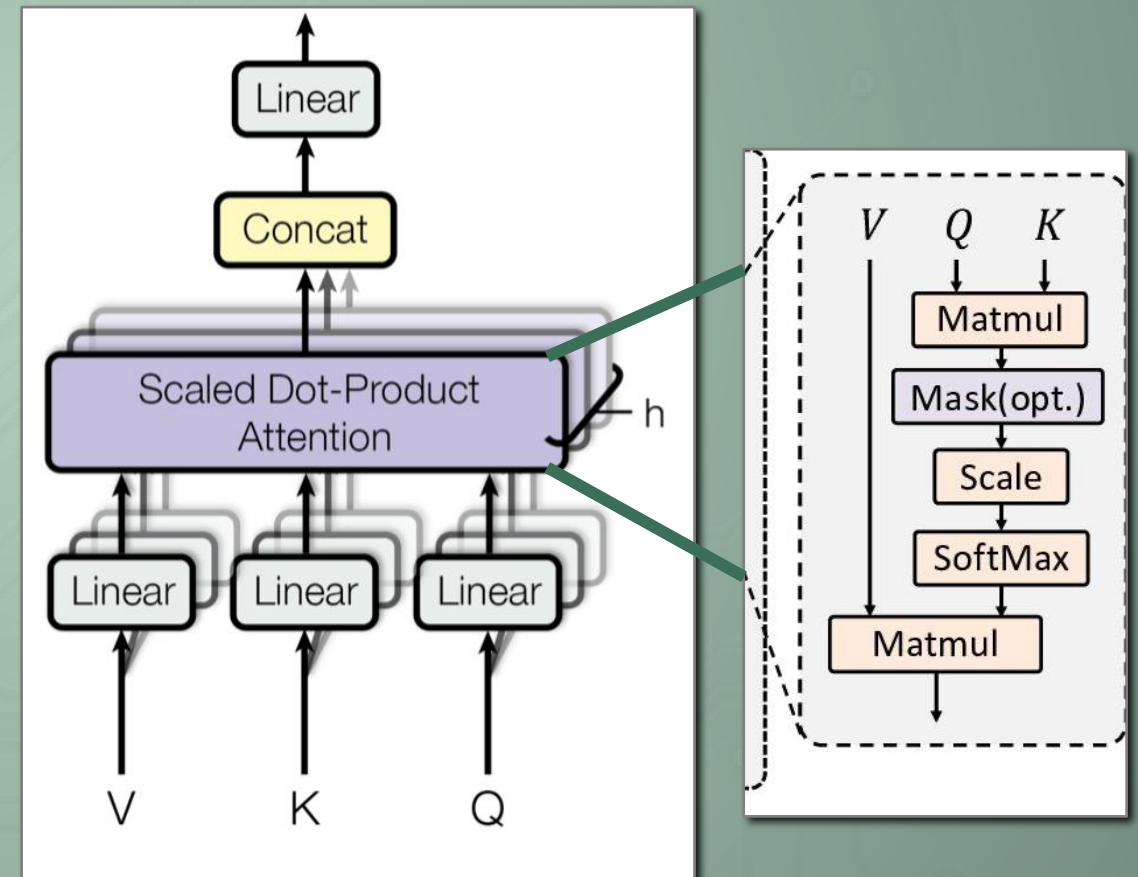
- All heads operate simultaneously
- Captures various data relationships at once

Linear Projections

- Input projected into subspaces for each head
- Allows focus on different input aspects

Concatenation and Linear Transformation

- Outputs from all heads concatenated
- Linearly transformed for final output



Transformer Block including Feed-forward Neural Network

... combines multi-head attention with feed-forward neural network, allowing model to process complex relationships in data

Multi-Head Attention

- Allows model to focus on different parts of the input sequence simultaneously
- Captures various types of relationships within data

Feed-Forward Neural Network

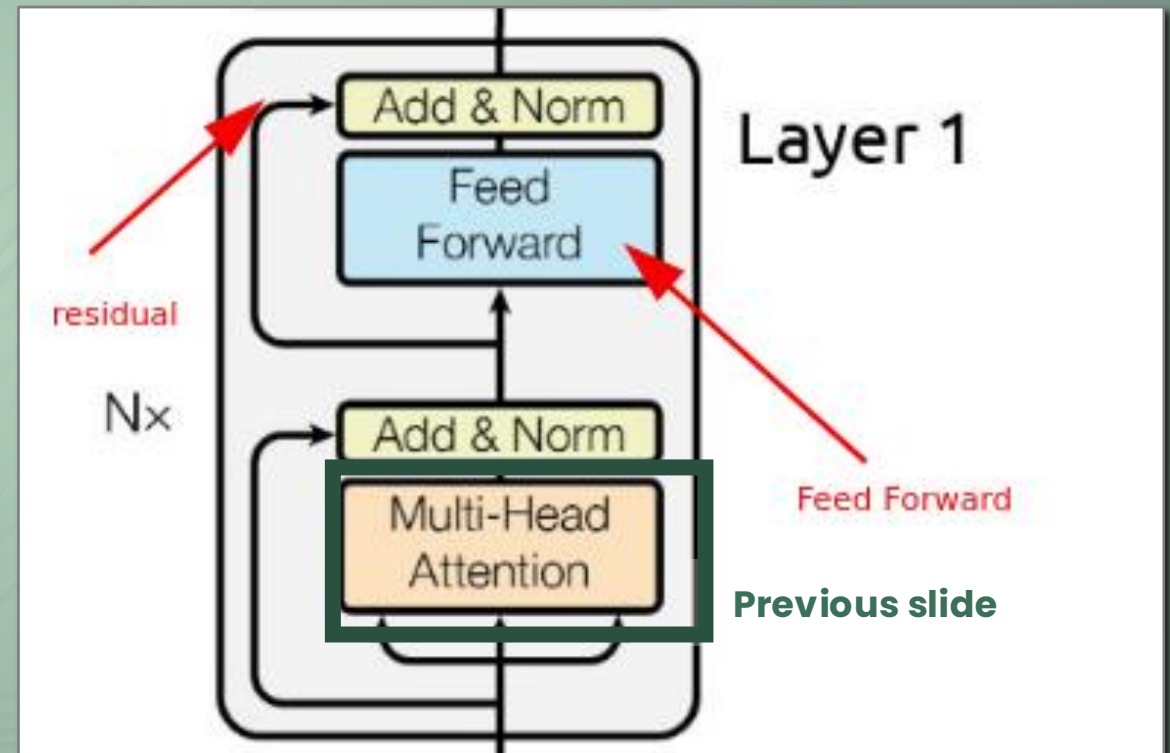
- Fully connected layer that processes output of attention layer
- Adds non-linearity and increases model's capacity

Layer Normalization

- Applied after each sub-layer to normalize activations
- Stabilizes learning process and allows for deeper networks

Residual Connections

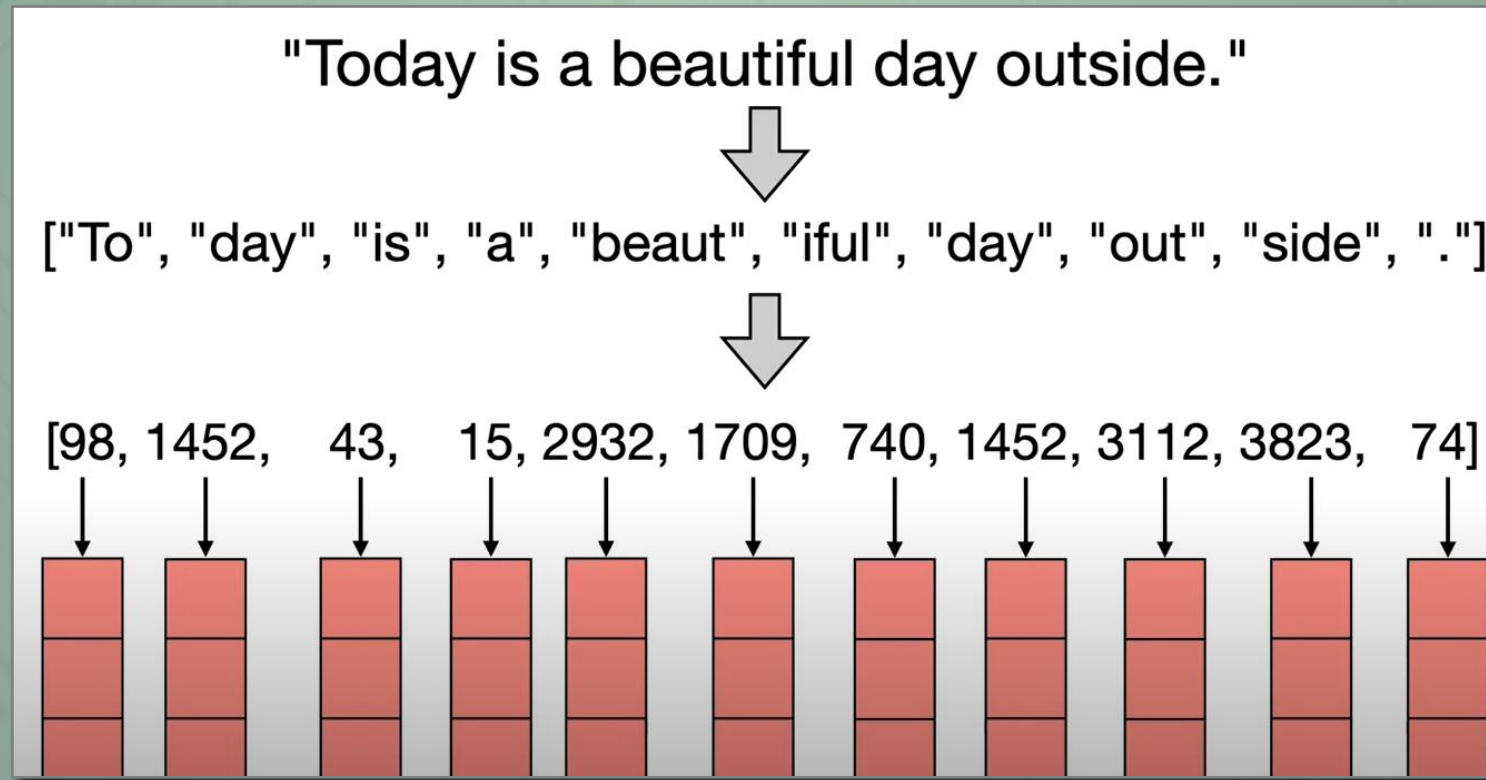
- Skip connections that allow gradients to flow more easily through the network
- Mitigates vanishing gradient problem in deep neural networks



Attention is all you need (2017)

Tokenizer

... converts raw text into sequence of tokens that can be processed by machine learning model by splitting the text into words or subwords, assigning each a unique numerical identifier



Tokenizer: Byte Pair encoding

... is subword tokenization algorithm for text compression and processing

Phase I: Learning

1. Initialize with character vocabulary
2. Count token pair frequencies
3. Merge most frequent pair
4. Add new merged token to vocabulary
5. Replace occurrences in corpus
6. Repeat 2-5 until reaching target vocabulary size or iteration limit
7. Finalize vocabulary and encoding rules

Phase II: Tokenization

1. Start with input text
2. Apply learned merge rules in order
3. Split text into subword units
4. Replace subwords with corresponding token IDs
5. Output sequence of token IDs

Byte Pair Encoding (BPE)

Phase I: Learning Learns common subwords

Large corpus of text

BPE Learning
Algorithm

Vocabulary

Phase II: Tokenization Splits words into tokens

Input words

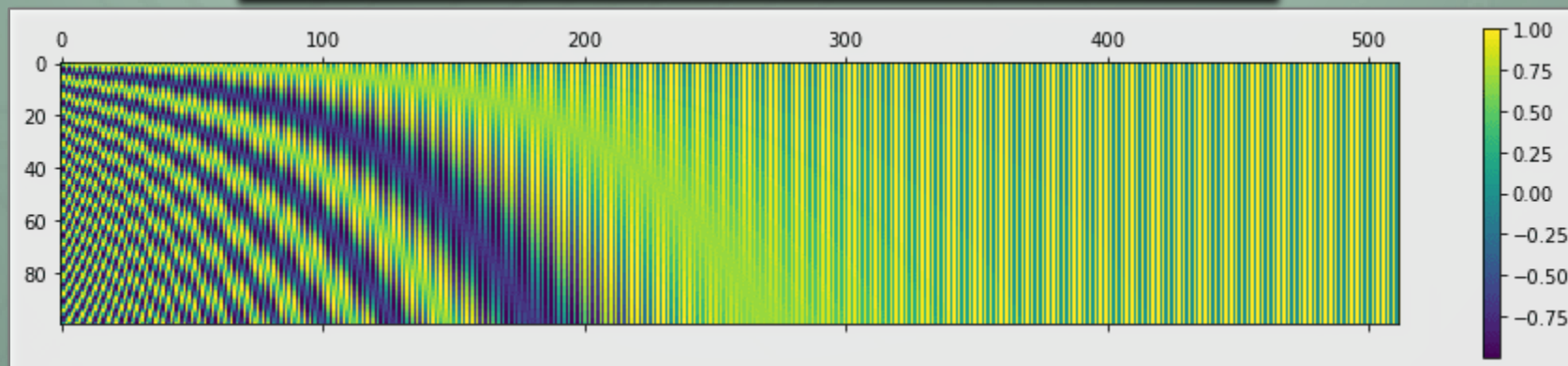
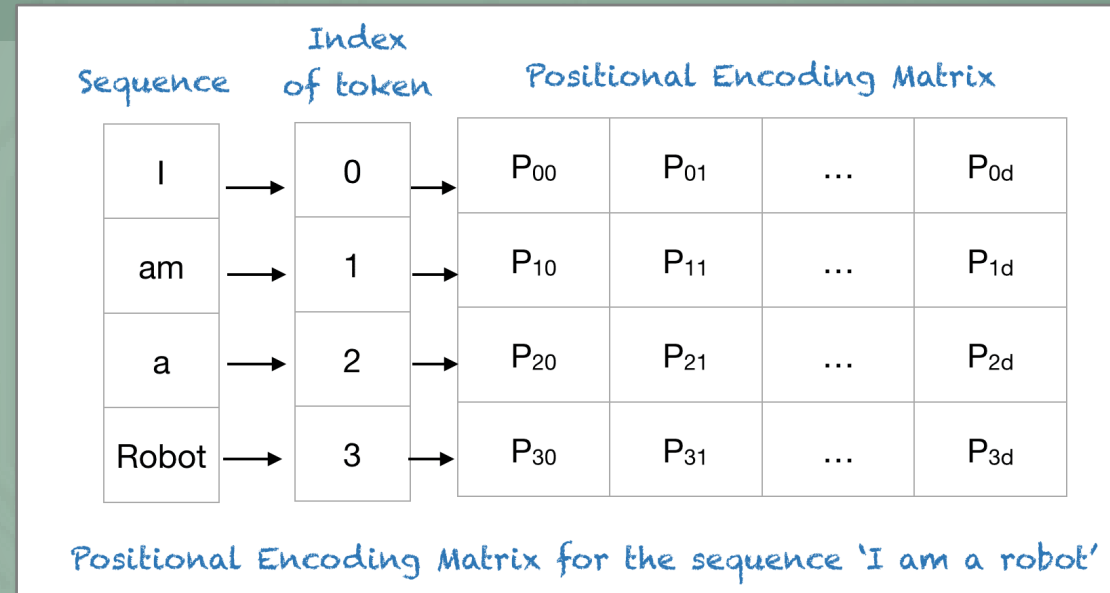
BPE Tokenization
Algorithm

← Vocabulary

Tokens

Positional Encoding

... provides sense of position for input tokens. Essential for models like Transformers that do not inherently process sequences in order. Overlapping trigonometric functions are used

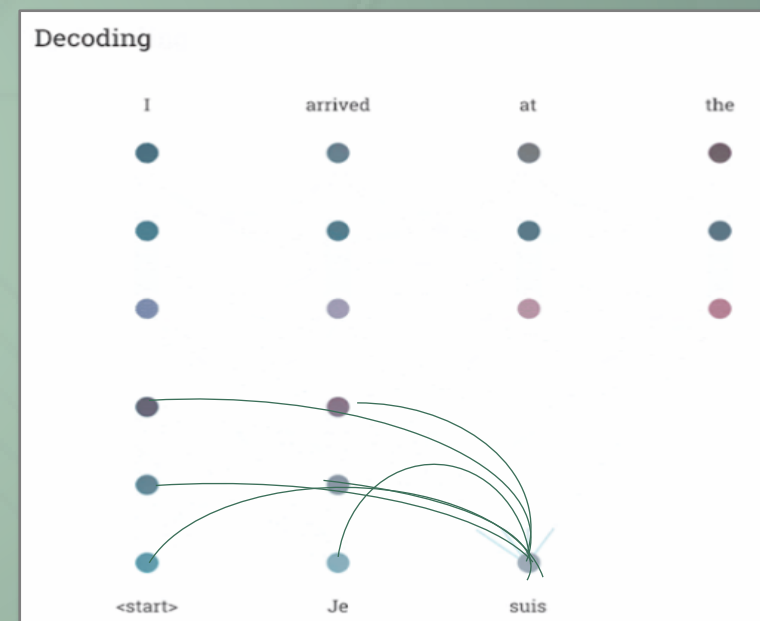
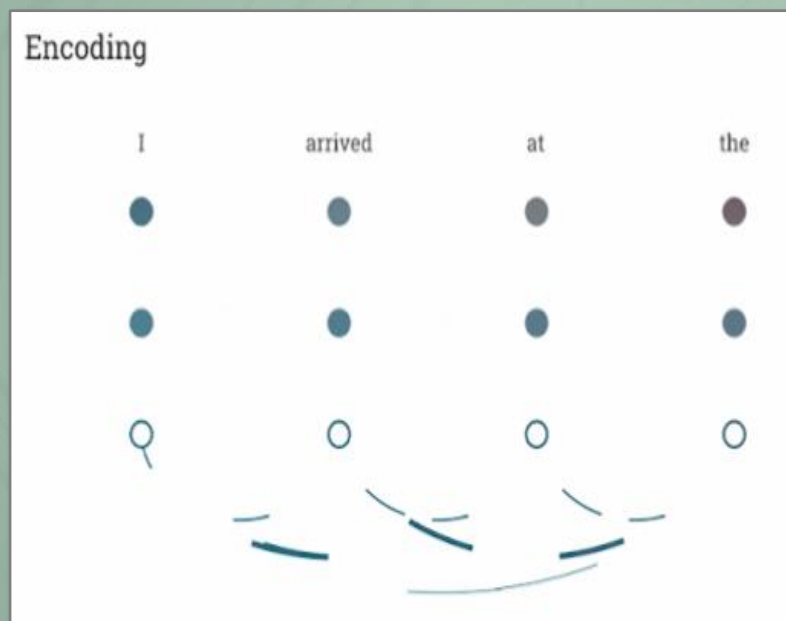
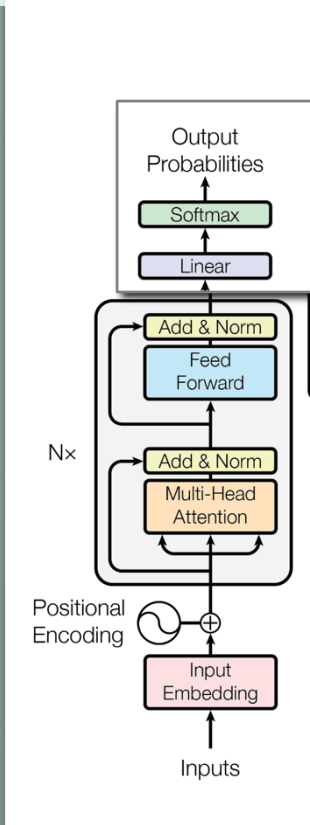


Difference between encoder, decoder

Initially, i.e. in 2014, transformer based translators have used both encoders and decoders. Encoder connects words (tokens) in both directions, decoder predicts next word.

Encoder:

- Sentiment Analysis
- Text classification



Training & Finetuning BERT for Sentiment Analysis

... Bidirectional Encoder Representations from Transformers (BERT) is a groundbreaking NLP model developed by Google. It classifies text data.

Training BERT from Scratch (Optional & expensive)

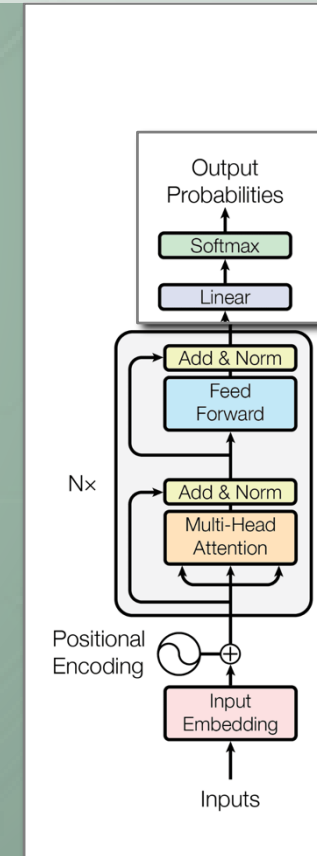
- Corpus Preparation: Gather a large, diverse corpus of text data.
- Pre-training Tasks: Masked Language Model (MLM) and Next Sentence Prediction (NSP).
- Computational Requirements: High, due to complexity and size of the model
→ better use HuggingFace

Finetuning Pre-trained BERT

- Objective: Adapt BERT to accurately classify sentiments in text (positive, negative, neutral).
- Dataset: Utilize sentiment-labeled dataset specific to the target domain (e.g., product reviews, social media posts, movie comments).
- Process:
 - Modify BERT's final layer to output sentiment categories.
 - Train model on sentiment dataset, adjusting parameters for optimal performance.

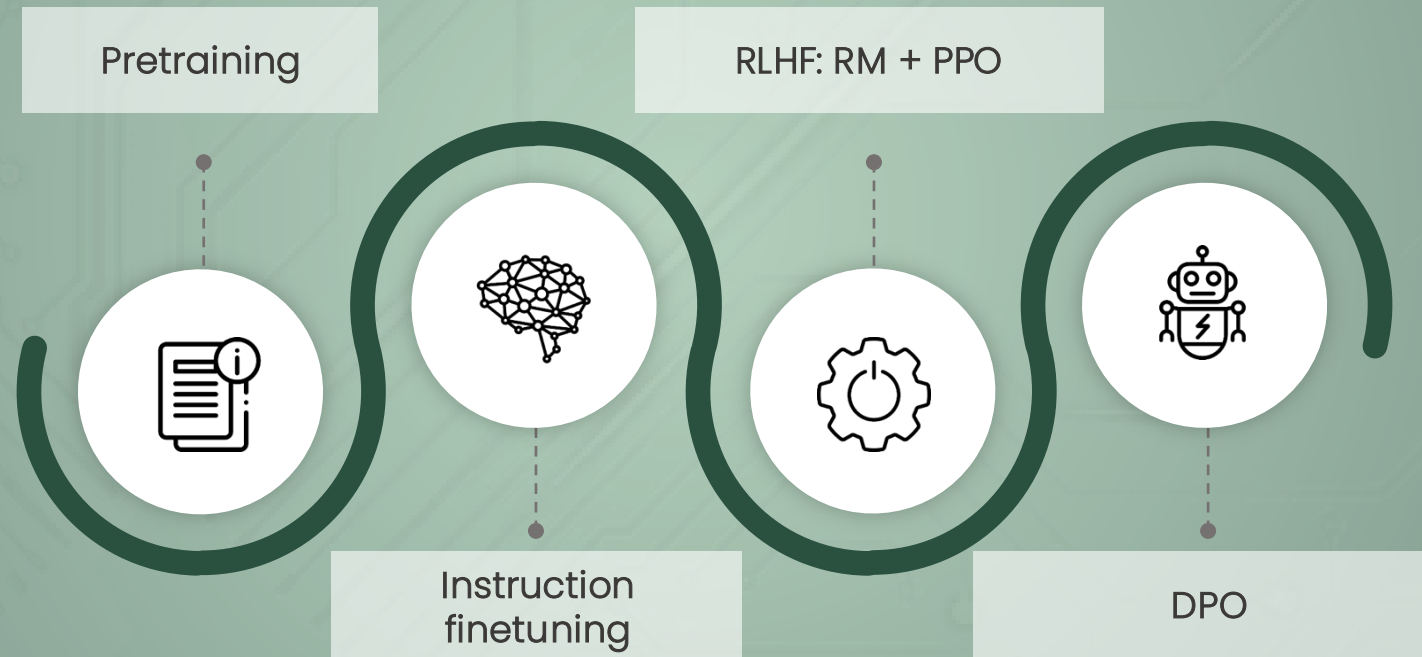
Encoder:

- Sentiment Analysis
- Text classification



Training Large Language Models

... involves feeding massive amounts of text data into neural networks with billions of parameters to enable model to generate human-like text across wide range of tasks and domains



Pretraining of Large Language Models

... involves exposing model to vast amounts of diverse text data, allowing it to learn general language patterns and representations through predicting masked words or next sentences



Massive Datasets

- Web pages, books, articles
- Diverse topics and styles
- Multiple languages



Self-Supervised Learning

- Predict next token
- Masked language modeling
- No human labeling required



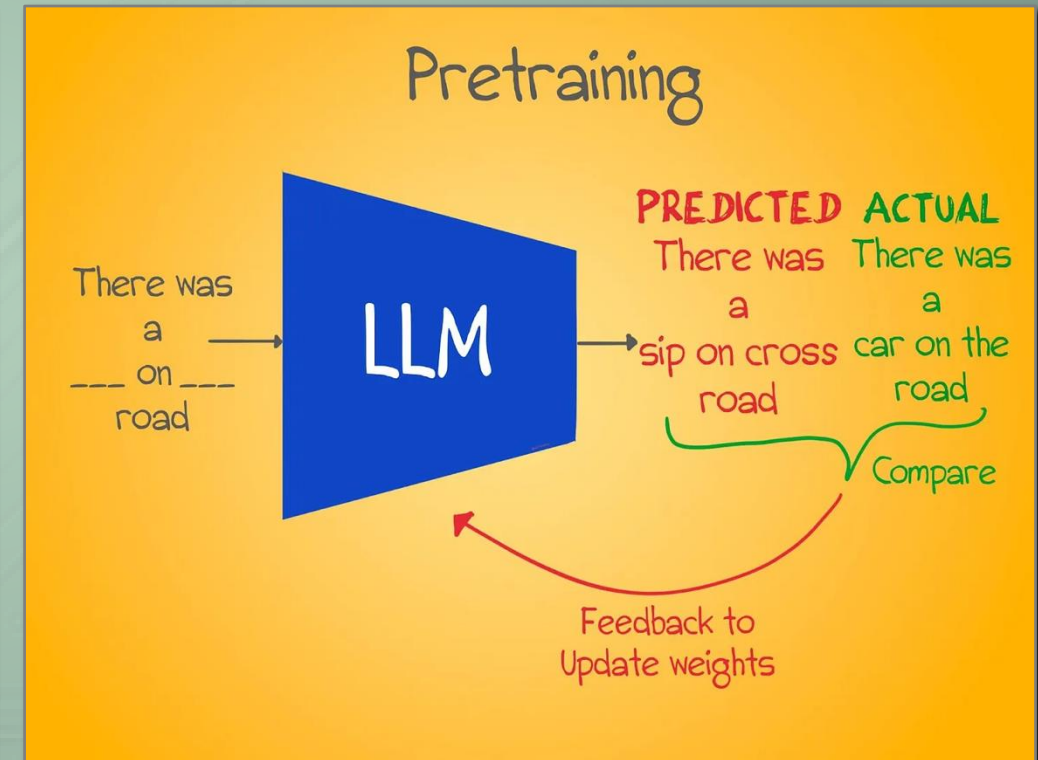
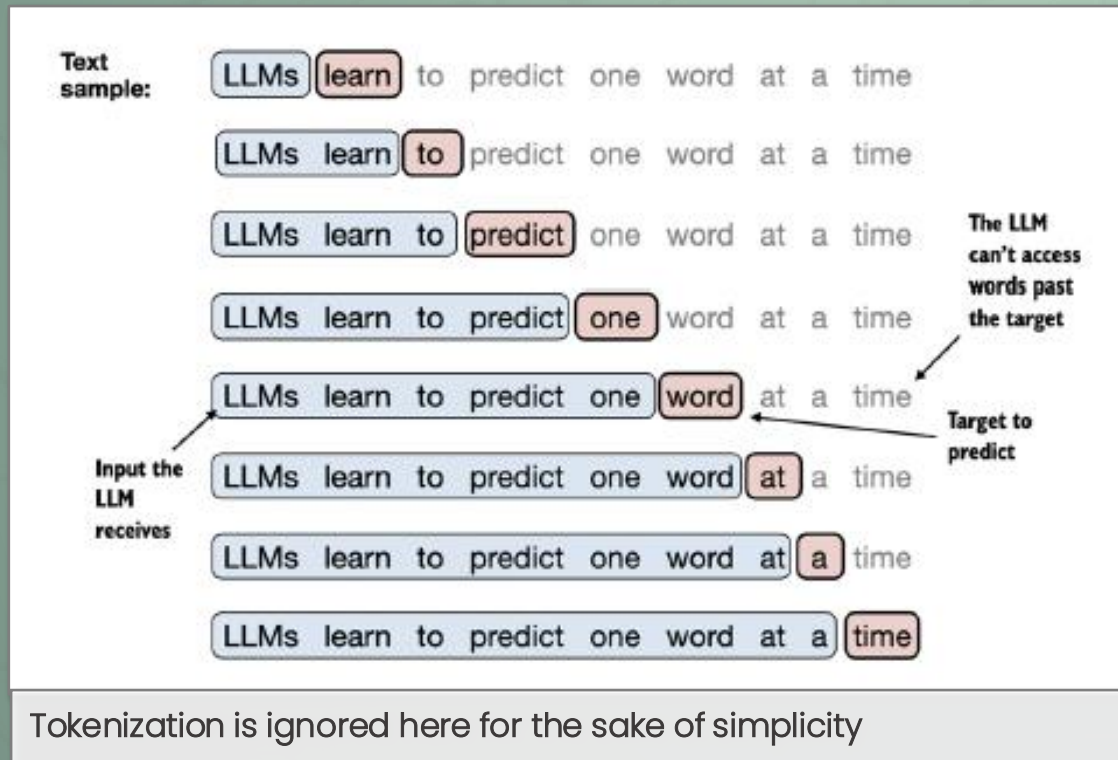
Massive Compute

- GPU/TPU clusters
- Distributed training
- Weeks or months of training



Pretraining of Large Language Models

Each sentence of words is predicted by previous (masked attention) words. Feedback is given whether prediction was correct or not. Based on this weights/parameters are updated.



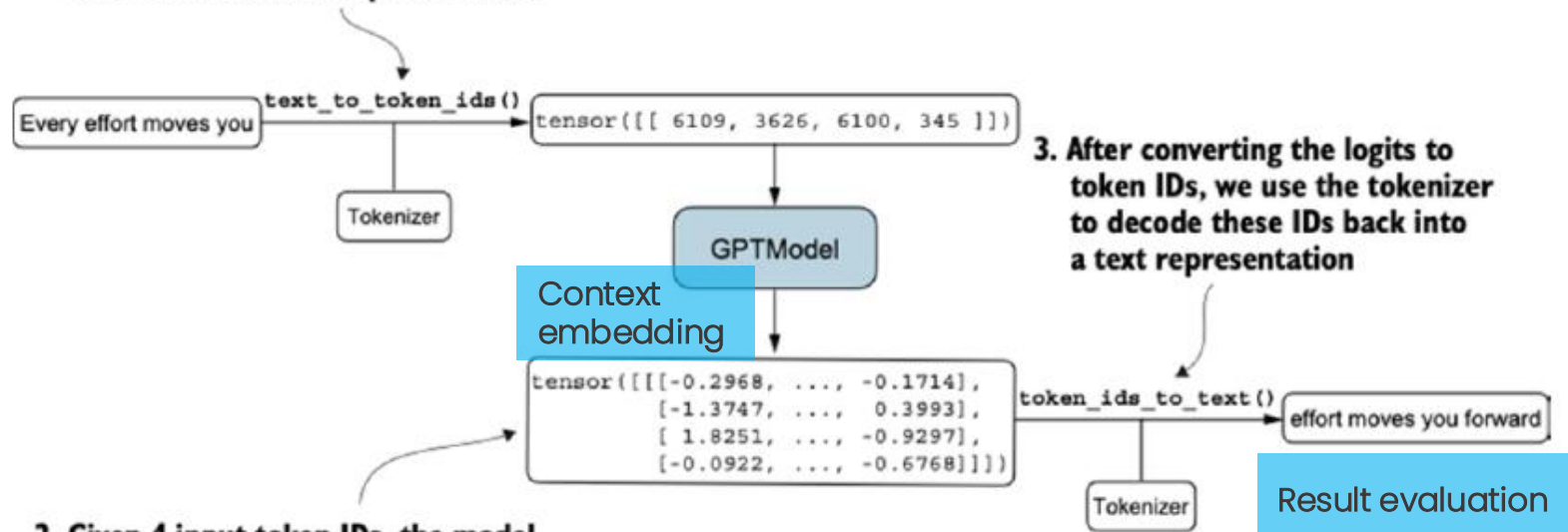
Training process: Pretraining

“Every effort moves you ” ... “forward”

Pretraining

1. Prompt-String is tokenized
2. position-embedded
3. context-embedded through GPTModel
4. most probable continuation token is added
5. tokens are detokenized
6. result is evaluated, LLM weights eventually updated

1. Use the tokenizer to encode input text into a token ID representation



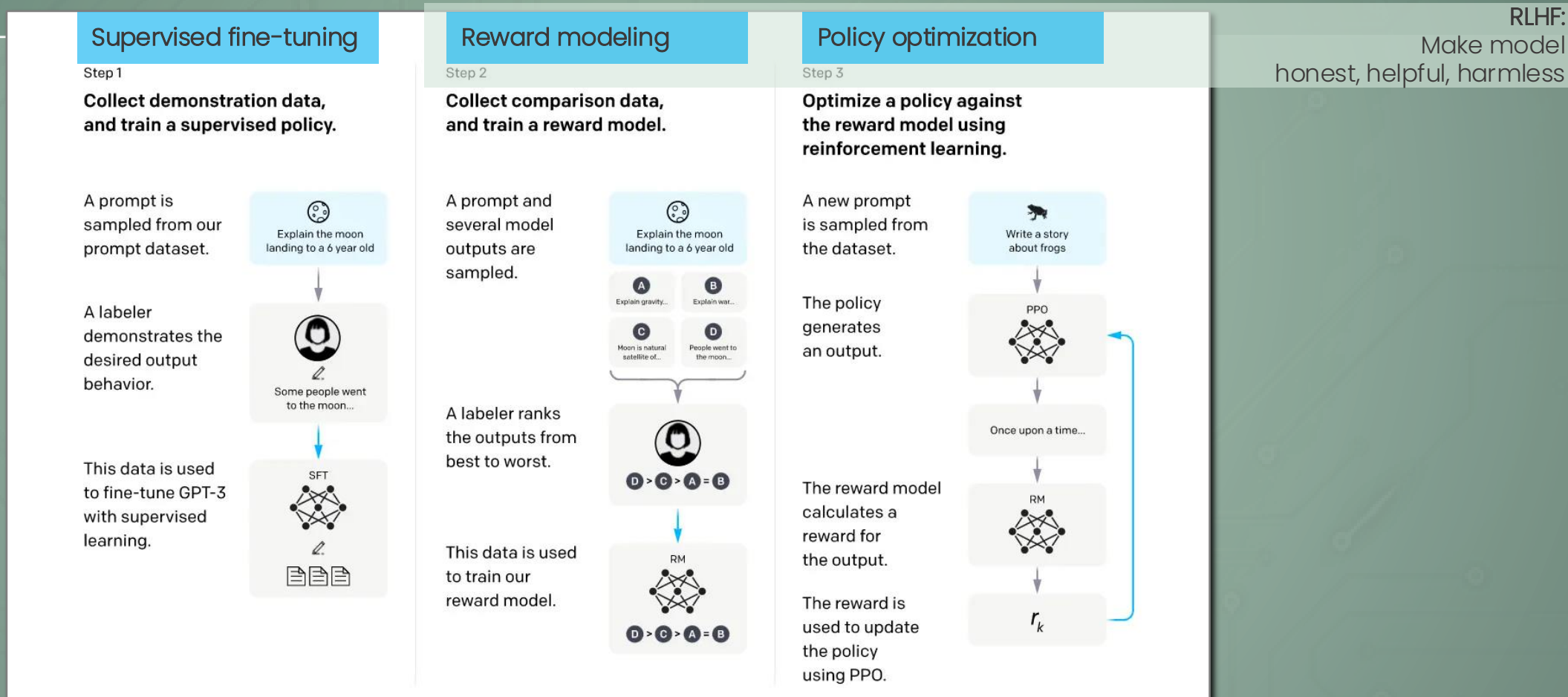
3. After converting the logits to token IDs, we use the tokenizer to decode these IDs back into a text representation

2. Given 4 input token IDs, the model produces 4 logit vectors (rows) where each vector has 50,257 elements (columns) equal to the vocabulary size

Result evaluation

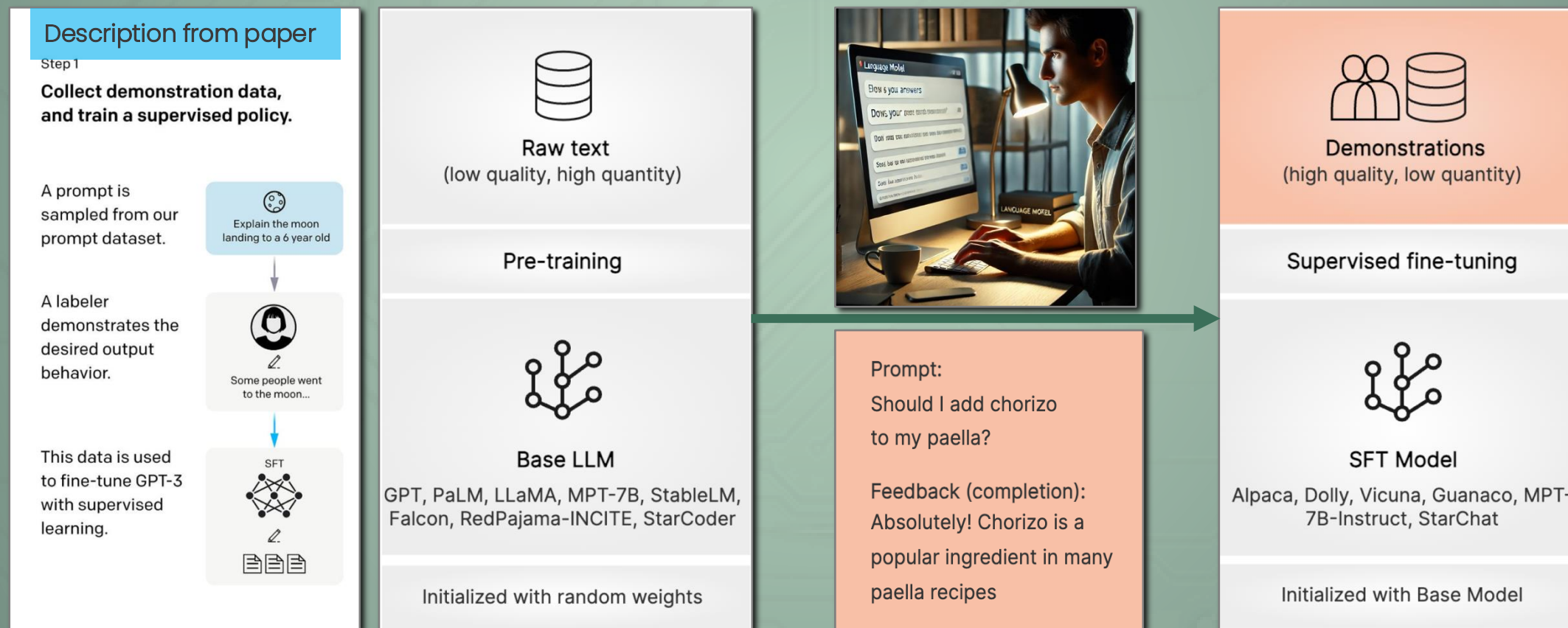
Training process: Alignment (SFT + RLHF)

Pretrained language model undergoes three-step process using supervised fine-tuning, reward modeling and policy optimization



Step 1: Instruction Finetuning

... transforms Base LLMs into question-answering machines by training them on instruction-response pairs, enabling them to respond to wide range of prompts/tasks accurately



Reinforcement Learning from Human Feedback (RLHF)

... is about making language models honest, helpful, harmless (3H)

- used to align language models with human preferences
- improving their ability to generate more helpful, accurate, and contextually appropriate responses
- across a wide range of tasks and conversations
- Steps:
 - Reward modeling
 - Policy optimization

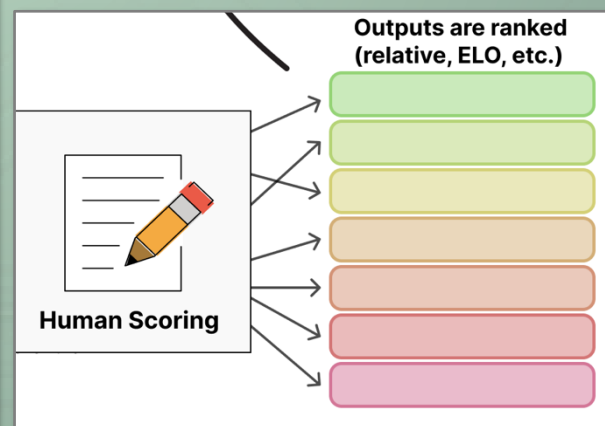
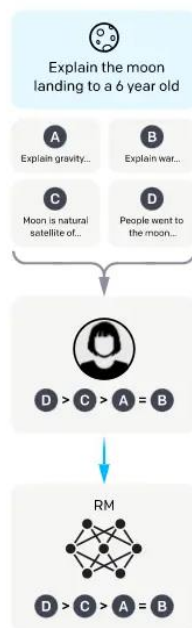
Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.



Step 3

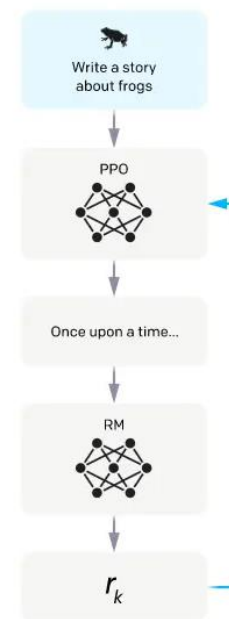
Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

The policy generates an output.

The reward model calculates a reward for the output.

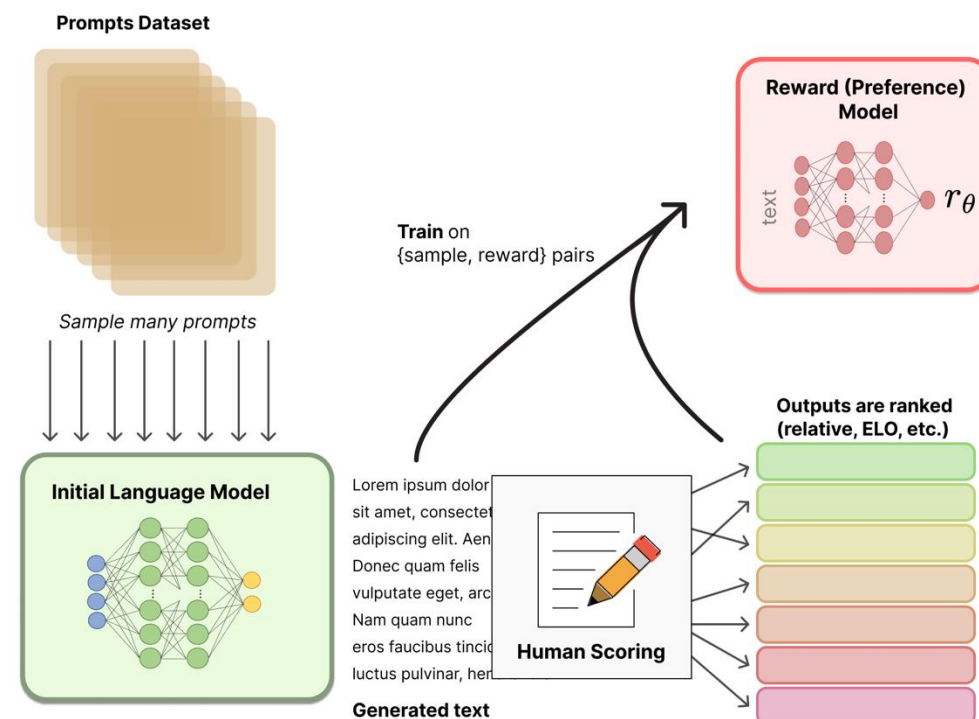
The reward is used to update the policy using PPO.



Step 2 (RLHF): Reward model

... is typically transformer-based neural network, similar in architecture to the language models it's designed to evaluate, e.g. smaller, obsolete LLM like GPT-2

- **Size:**
Generally smaller than main language model to be more computationally efficient.
- **Input:**
Takes in a prompt-response pair (or sometimes just the response, with the prompt as context).
- **Output**
Produces a single scalar value representing the predicted quality or preference score for the given input.
- **Training:**
Fine-tuned on a dataset of human-labeled preferences, often using comparative rankings (e.g., response A is preferred over response B for given prompt).
- **Adaptation:**
Can be iteratively updated as more human feedback data becomes available during the RLHF process.



Step 3 (RLHF): Proximal Policy Optimization

... optimizing Language Model Through Iterative Feedback and Policy Updates

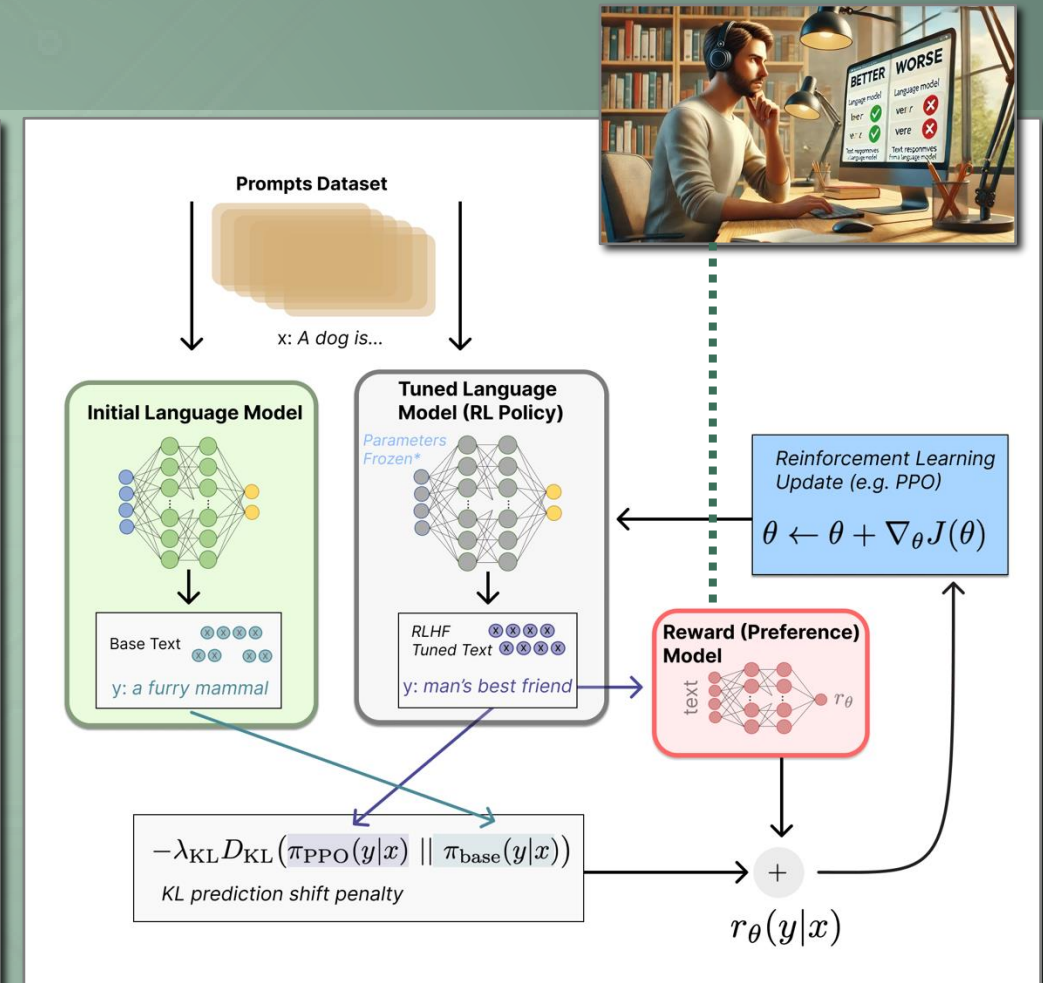
Proximal Policy Optimization (PPO)

- Simple Reinforcement learning algorithm for optimizing AI agent policies
- makes controlled updates to improve performance while avoiding drastic changes
- uses “clipped surrogate objective function”, balancing efficiency and stability

Process

1. Initial language model generates base responses
2. Tuned model (RL policy) produces refined outputs
3. Reward model evaluates based on human preferences
4. PPO algorithm updates policy to maximize rewards
5. Preventing “catastrophic forgetting”:
KL divergence penalty maintains consistency with base model, i.e. base model must not drift too much

→ Human feedback for reward model is ongoing!



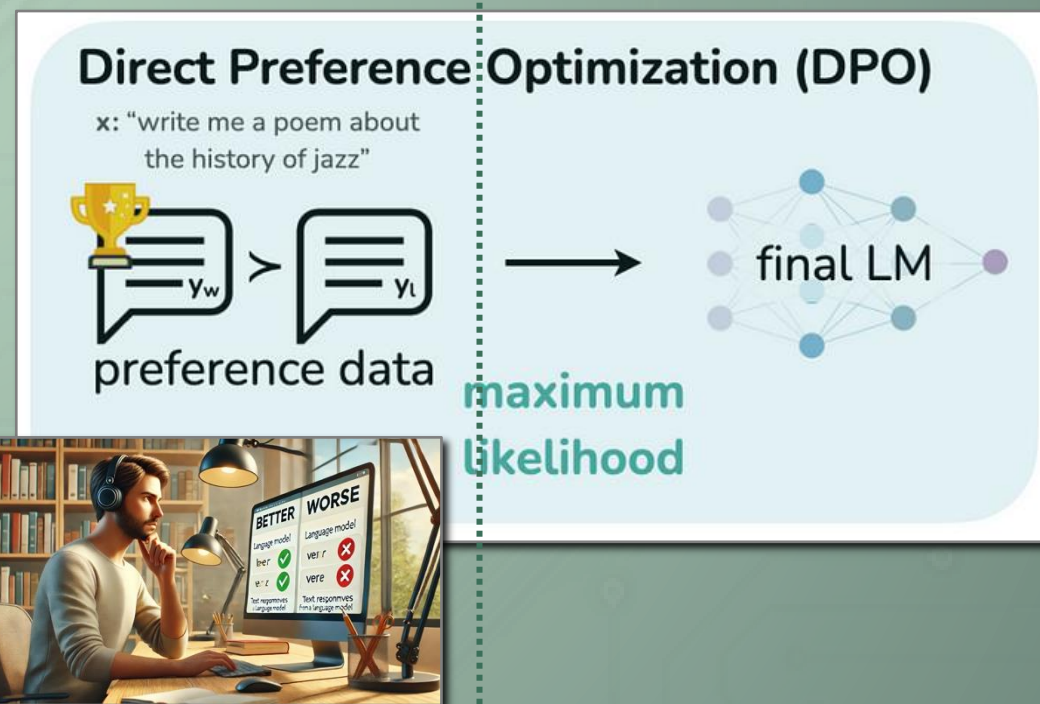
Training process: DPO

... offers streamlined alternative to complex RLHF pipelines, potentially revolutionizing preference-based language model training

Process

1. Starts with pre-trained, instruction finetuned language model.
2. Collects pairs of responses, where humans have indicated preference for one over the other.
3. Instead of training a separate reward model, DPO directly optimizes the language model to produce preferred outputs.
4. It uses loss function that encourages the model to assign higher probability to preferred responses.
5. Optimization process maintains a balance between improving on preferences and staying close to the original model's behavior.

→ Human feedback is collected beforehand, not in real-time during training

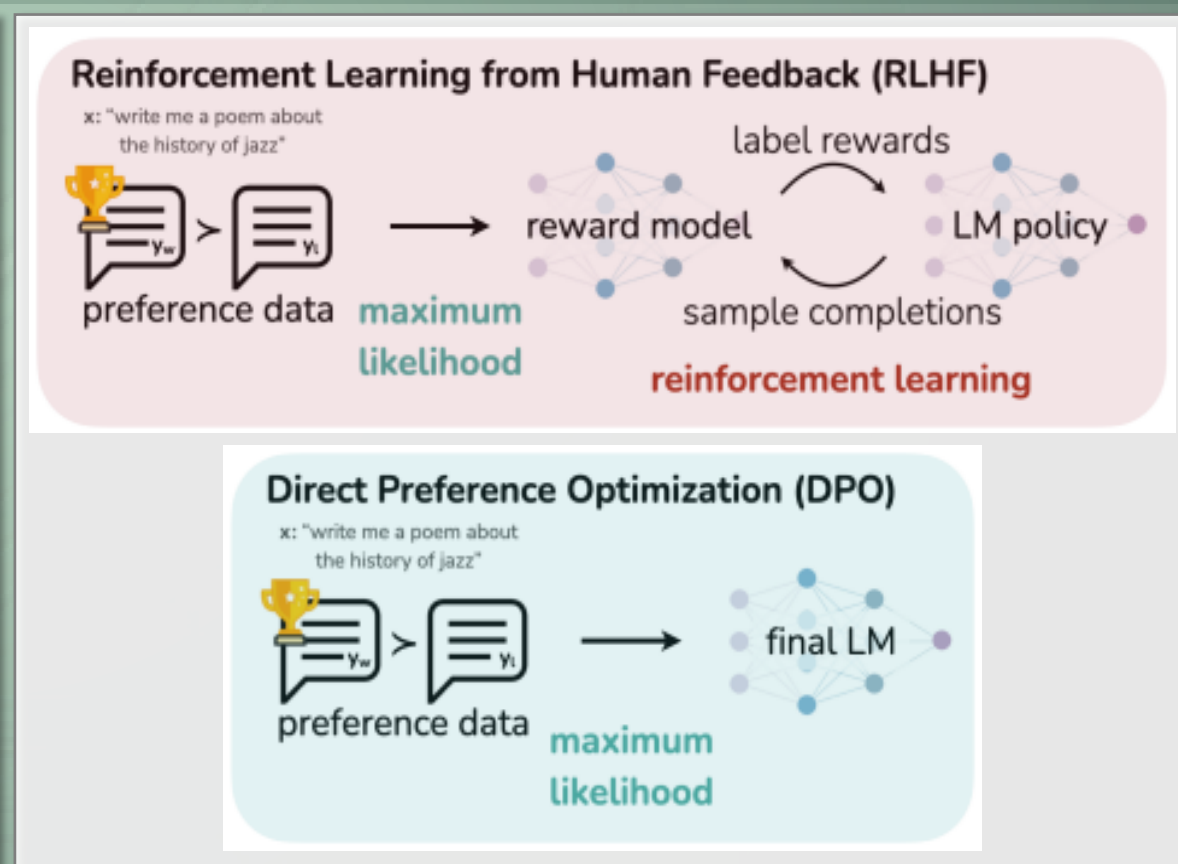


Comparison: RLHF/PPO vs. DPO

RLHF/PPO offers greater flexibility, adaptability at cost of computational complexity

DPO provides simpler approach that may sacrifice adaptability, fine-grained control.

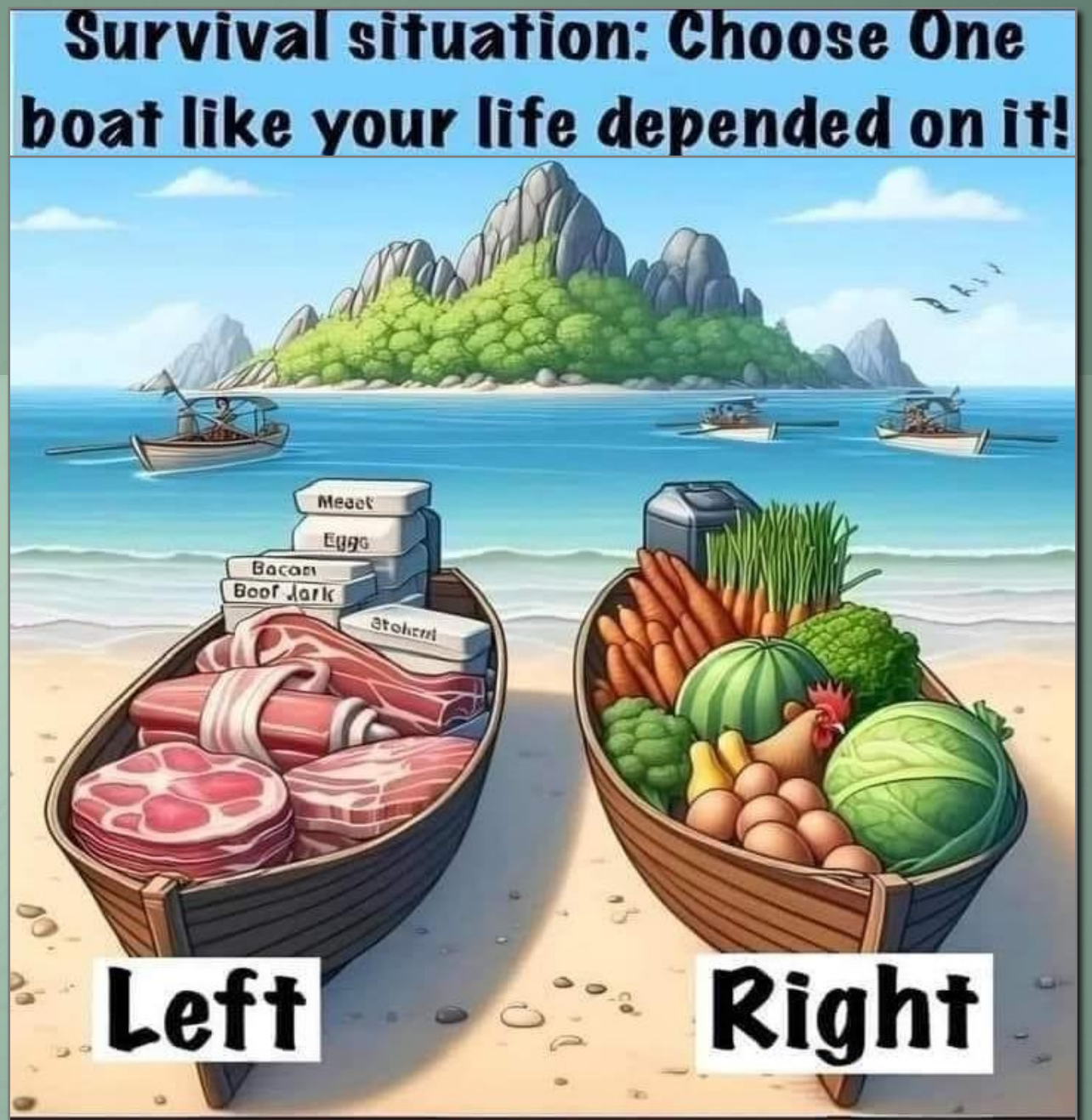
Aspect	RLHF with PPO	DPO
Approach	Simple Reinforcement Learning	Supervised Learning
Complexity	high	- Lower - however needs more data
Reward Model needed	yes	no
Training Process	Needs ongoing human feed-back loop	Uses precollected preference data
Advantages	- More flexibility - fine-grained control - Needs much less data	- Simpler, more stable implementation - easier to maintain
Examples	- OpenAI ChatGPT - Anthropic Claude	- Mistral - Meta Llama 3



RLHF/PPO, DPO

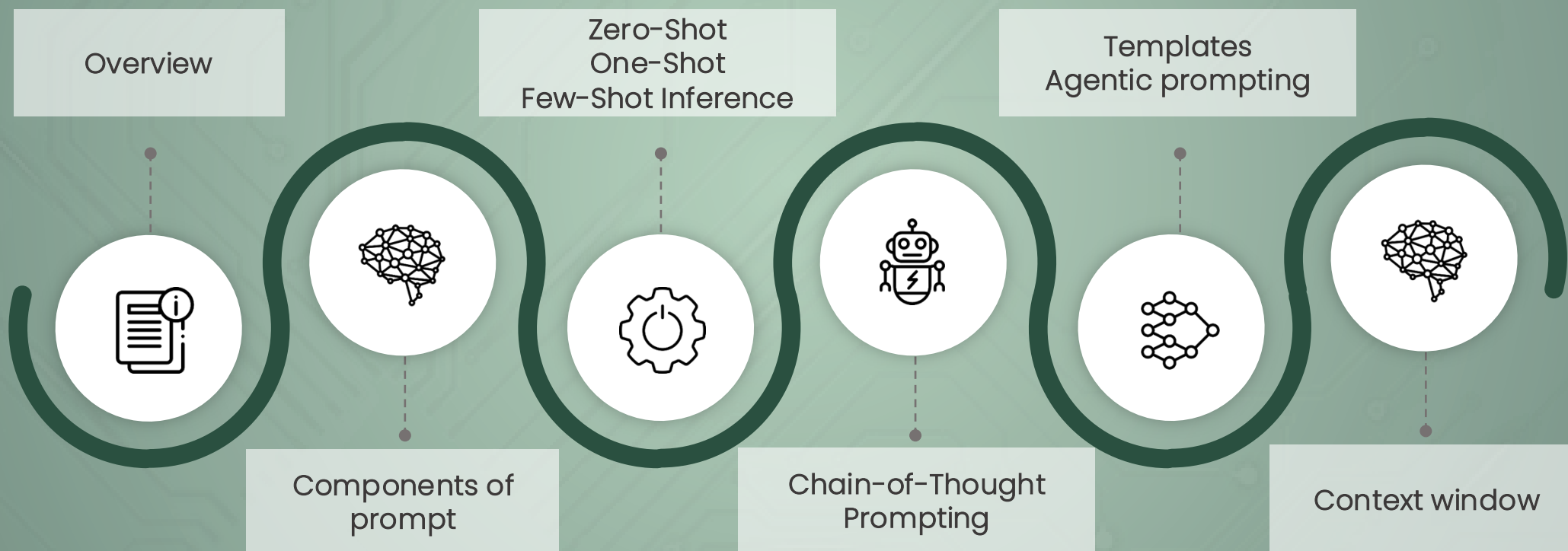
... face same challenge:
preferences are individual

- selection of “better” answer is often based on individual preferences
- No perfect solution, however as outcome LLM must be
 - Honest
 - Helpful
 - harmless



Data Augmentation: Prompt Engineering

... is the art and science of crafting precise and effective input prompts to elicit desired outputs from large language models, optimizing their performance for specific tasks or applications



Prompt Engineering Overview

What is it good for?

Improved Relevance, Specificity

- Guides model to generate content more closely aligned with the intended topic or purpose
- Encourages more detailed and precise outputs by providing clearer expectations
- Allows augmentation of new data beyond training set
- Allows to use roles, examples:
 - explain like you are pirate
 - Explain like I am 5 (ELI-5)

Consistency Control

- Helps maintain consistent tone, style, format throughout generated text.

Creativity Direction

- Can steer model's creative outputs in desired directions



Components of a Prompt

... include user input (query, task), instruction set (response guidelines), context (background information, examples), system prompt (overarching instructions shaping model's behavior).

Aspect	User Input	Instruction Set	Context	System Prompt
Description	- specific query or task - provided by end-user	Guidelines, context to direct model's response	Relevant background information, examples, data to inform model's output	Overarching instructions that shape model's behavior across all interactions
Example	"Where do migratory birds spend the winter?"	"Answer the following question and explain it as if the user was five years old."	Examples from a VectorDB search for closest document	You are an expert ornithologist specializing in bird migration patterns and winter habitats. Your role is to provide accurate, informative answers about where migratory birds spend their winters.
Purpose	Defines - primary question - task model needs to address	Shapes - Format - tone - complexity of model's output	Provides additional information to make the response more accurate or relevant	Sets overall tone, role, and boundaries for model's interactions
Potential Variations	Can range from - simple questions to - complex multi-part requests	May include formatting instructions, language preferences, or specific output requirements	Can include historical data, related examples, or domain-specific knowledge	Can define model's persona, ethical guidelines, specific domain expertise

Prompting Language Considerations

... use clear, simple language, be specific about expectations, maintain logical structure, include illustrative examples, and tailor prompt to AI system's capabilities

Use plain language

- Avoid jargon, complicated terms.
- Make instructions easy to understand so the AI quickly grasps intent.

Be specific

- Clearly state expectations, desired outcomes.
- Provide detailed information

Maintain logical structure

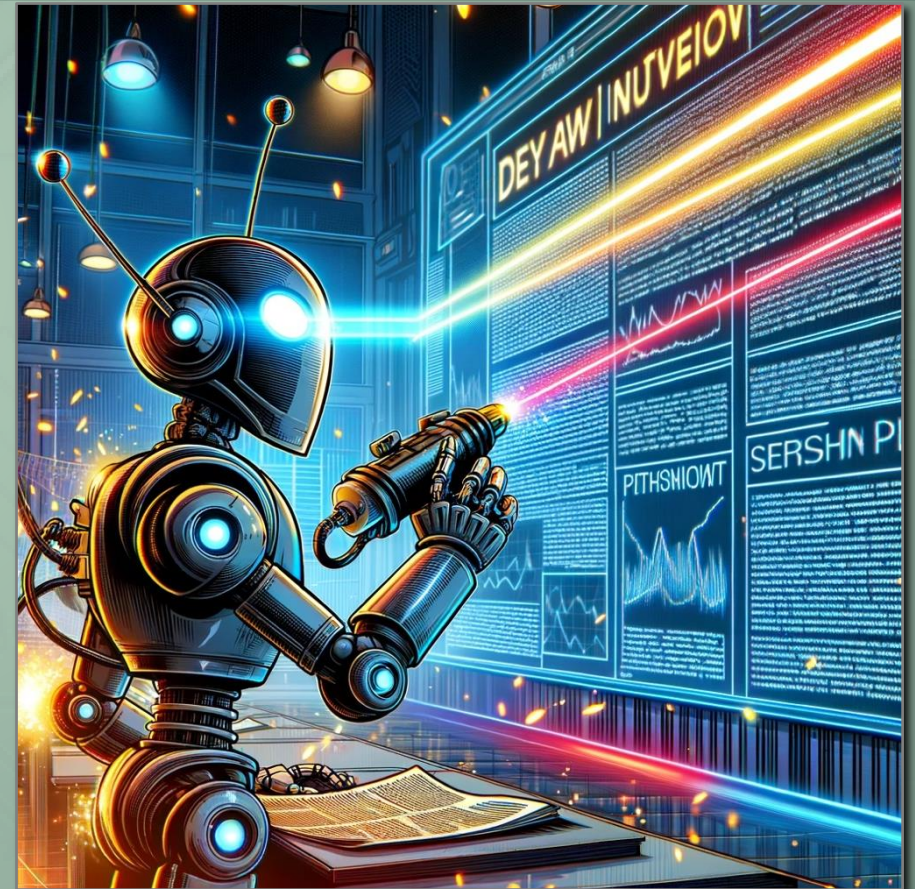
- Organize prompt in coherent manner

Include examples

- If possible, provide examples to illustrate expected type of output
- This helps AI generate more accurate results.

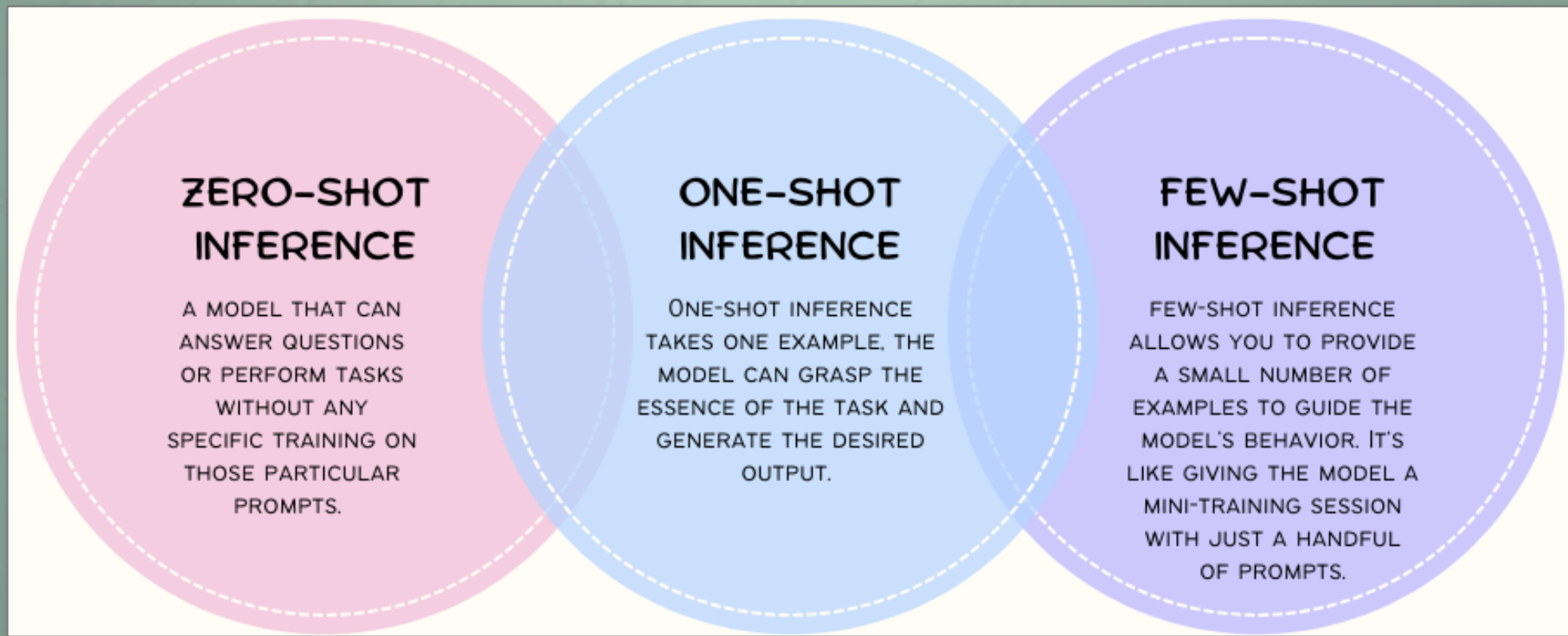
Consider context

- Tailor your prompt to the AI system and its capabilities.



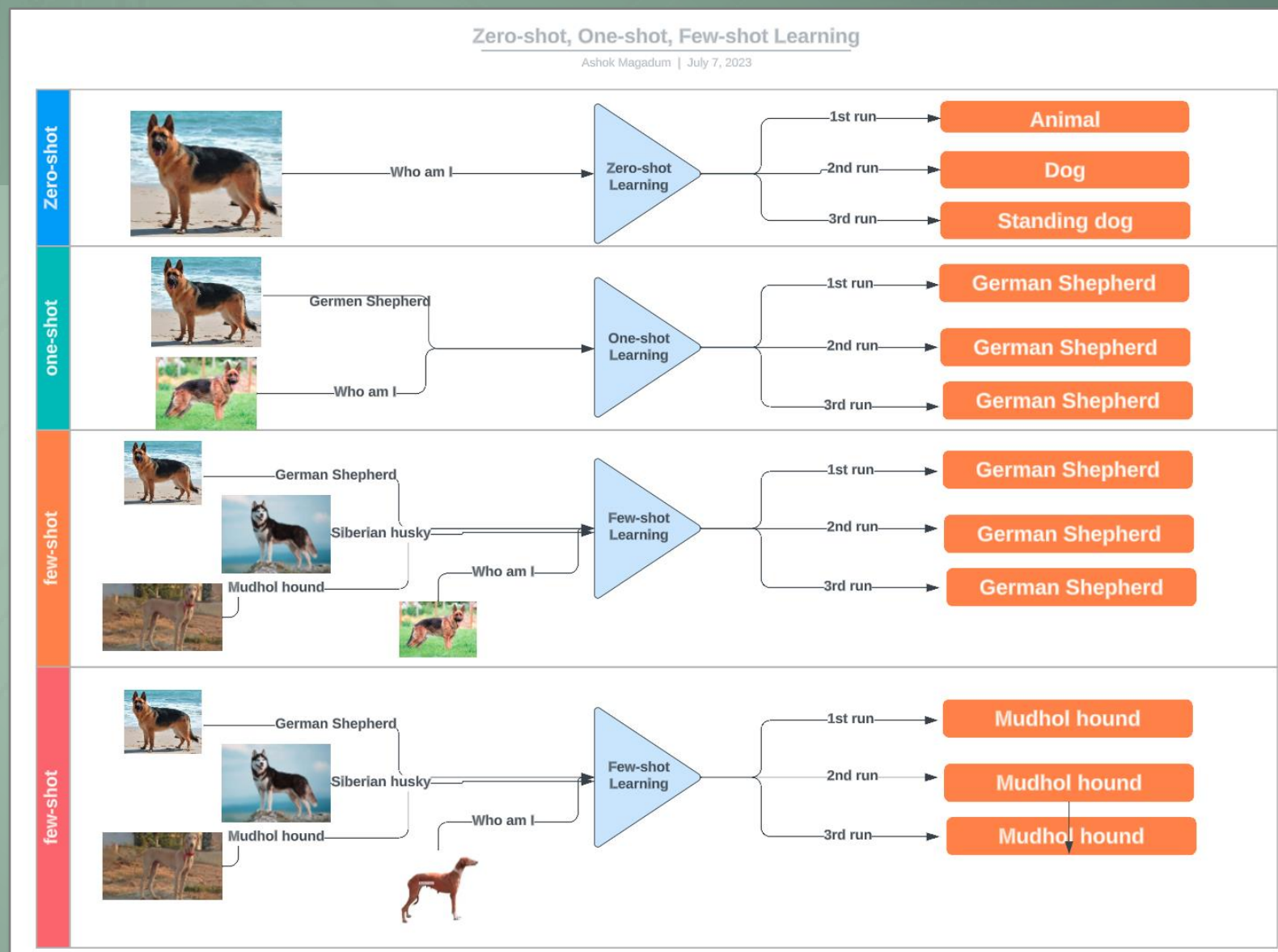
From Zero-Shot to few shot Prompting

Basically LLM is zero-shot learner, i.e. it transfers knowledge to new contexts. However by adding one or few examples performance significantly increases.



From Zero-Shot to few shot Prompting

... Example of increasing precision through examples



Chain-of-Thought Prompting

... is technique in natural language processing where model generates intermediate reasoning steps to improve accuracy and coherence of its final output

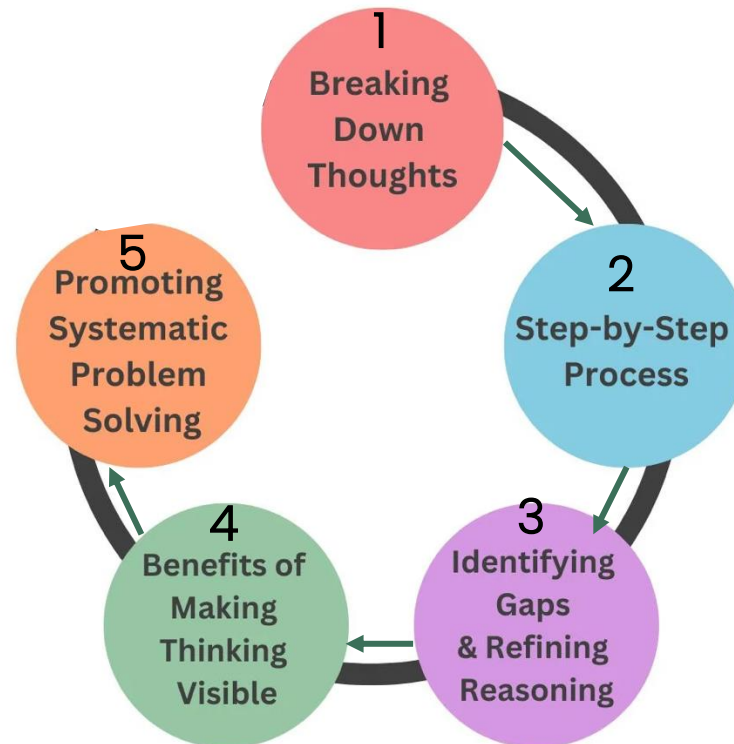
5. Promoting Systematic Problem-Solving

- Technique fosters systematic and thorough approach to problem-solving.
- It encourages individuals to consider multiple viewpoints. It also aids them in exploring alternative solutions.

4. Benefits of Making Thinking Visible

- CoT Prompting makes thought process visible and explicit.
- This helps individuals better understand their own reasoning.
- It also enables them to communicate their ideas more effectively to others.

How Chain of Thought Prompting Works



1. Breaking down thoughts

- Chain of Thought (CoT) Prompting encourages people to dissect their thought processes into smaller, connected steps.
- This method helps reveal fundamental logic, reasoning behind their ideas.

2. Step-by-step Process

- Approach begins by presenting initial question or issue.
- Participants then divide thoughts into manageable segments.
- Each step is clearly expressed using linking phrases such as "because," "therefore," or "as a result."

3. Identifying Gaps & Refining Reasoning

- As individuals progress through process, they may identify gaps in their reasoning.
- They can gather additional information to refine their thoughts and strengthen their arguments.

Chain-of-Thought Prompting Example

Language models cannot contemplate logically. Chain-of-thought helps simulate logical thinking

Prompt

Solve the following word problem step by step:

"A bakery sold 136 loaves of bread on Monday. On Tuesday, they sold 42 more loaves than on Monday. On Wednesday, they sold half as many loaves as they did on Tuesday. How many loaves of bread did the bakery sell in total over these three days?"

Please show your reasoning for each step.

Expected Response:

Step 1: Calculate the number of loaves sold on Tuesday

- Monday sales: 136 loaves
- Tuesday sold 42 more than Monday
- Tuesday sales = $136 + 42 = 178$ loaves

Step 2: Calculate the number of loaves sold on Wednesday

- Wednesday sold half as many as Tuesday
- Wednesday sales = $178 \div 2 = 89$ loaves

Step 3: Sum up the total loaves sold over three days

- Total = Monday + Tuesday + Wednesday
- Total = $136 + 178 + 89 = 403$ loaves

Therefore, the bakery sold a total of 403 loaves of bread over these three days.

Template-Based Prompting

... is structured approach to crafting AI prompts using pre-defined formats or patterns to ensure consistency and effectiveness in AI interactions

Benefits of Template-Based Prompting

- **Consistency:**
Ensures a uniform structure across multiple prompts
- **Efficiency:**
Streamlines the prompt creation process
- **Clarity:** Provides clear instructions to the language model
- **Customization:**
Allows for easy adaptation to various tasks and contexts
- **Reproducibility:**
Facilitates consistent results across different runs

Python example of prompt template

```
sentiment_template = PromptTemplate(
    "Analyze the sentiment of the following text:\n\n" +
    "Text: {text}\n\n" +
    "Provide your analysis in the following format:\n" +
    "Sentiment: [Positive/Negative/Neutral]\n" +
    "Confidence: [0-100]%\n" +
    "Key phrases: [List key sentiment indicators]"
)
```

Using the template

```
text = "I absolutely loved the new restaurant! The food was amazing."

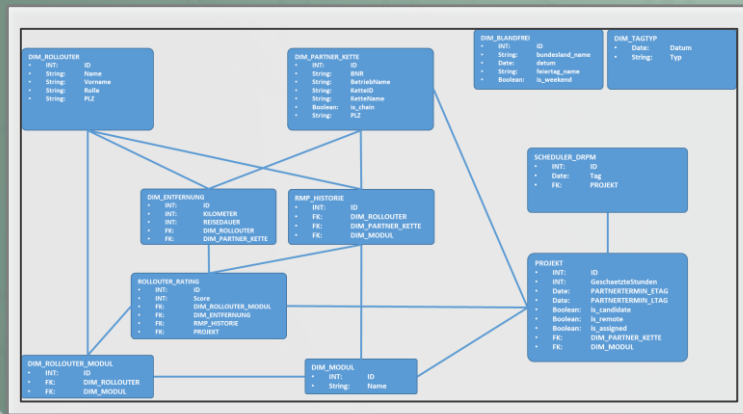
prompt = sentiment_template.format(text=text)

print(prompt)
```

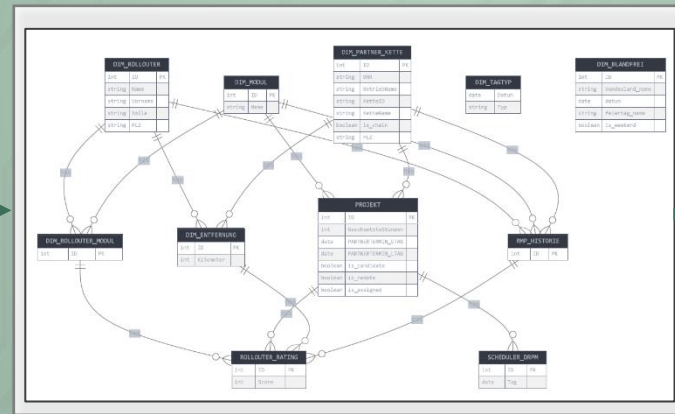
Agentic prompting

Example: Business process automatization with Java Spring Boot can be generated by agentic pipeline in Python, i.e. consecutive prompts that use results of previous steps

Entity-Relation diagram (PowerPoint)



Entity-Relation diagram (technical)



Backend (Java Spring Boot) With different layers

CONTROLLER: REST API

SERVICE: Business logics

REPOSITORY: Technical

DTO: Technical

MODEL: Database

Agentic prompting

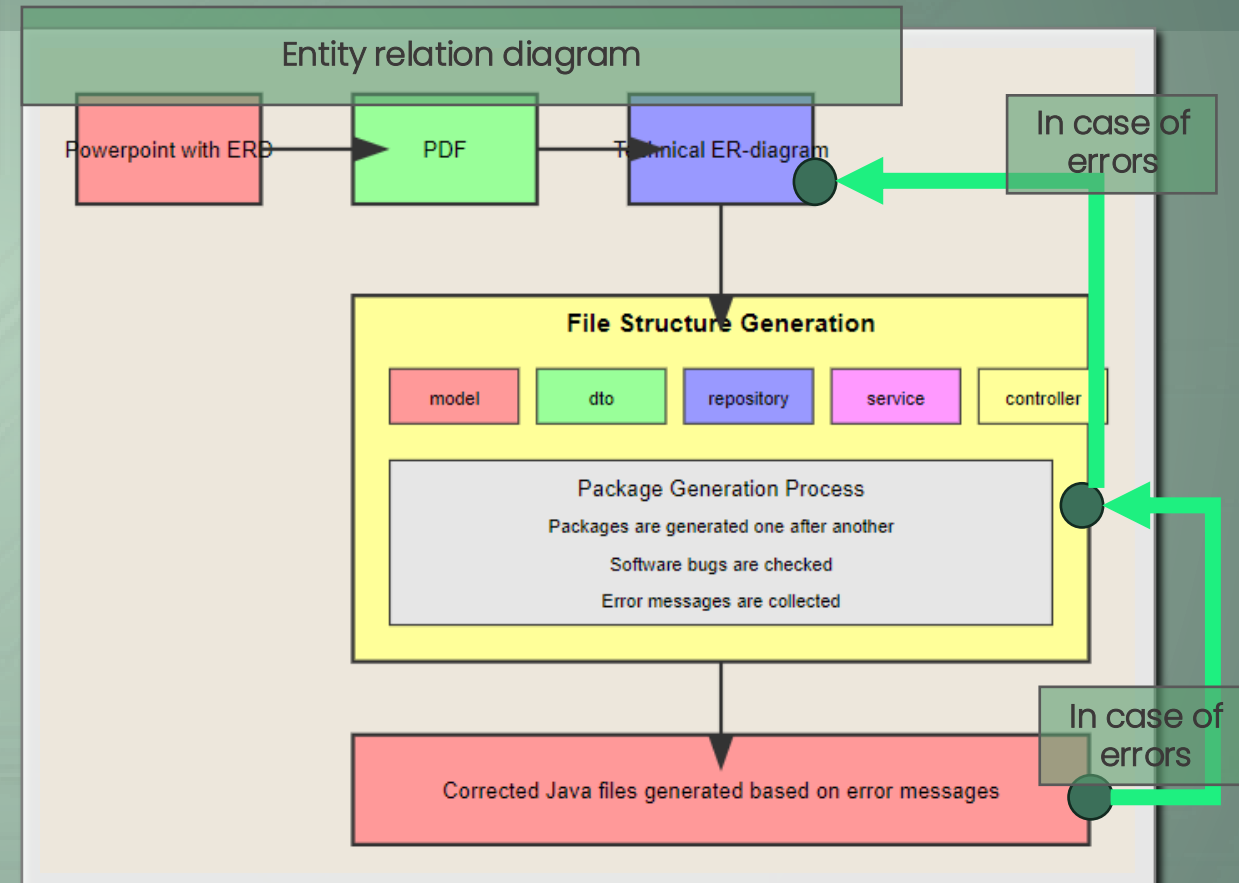
Example: Business process automatization with Java Spring Boot can be generated by agentic pipeline in Python, i.e. consecutive prompts that use results of previous steps

Python controls following

- Entity Relation diagram is made by hand in Powerpoint
- Powerpoint is saved in PDF and transformed by LLM to technical ER diagram
- Java Spring Boot file structure is generated
- Based on it all Java packages are generated
- All files are saved at correct position
- If bugs occur they are put into LLM, which then generates corrected files
- Finally program runs correctly

Problem with agentic pipelines

- results are not 100% correct.
- Therefore at each step results must be checked, step must be repeated eventually.

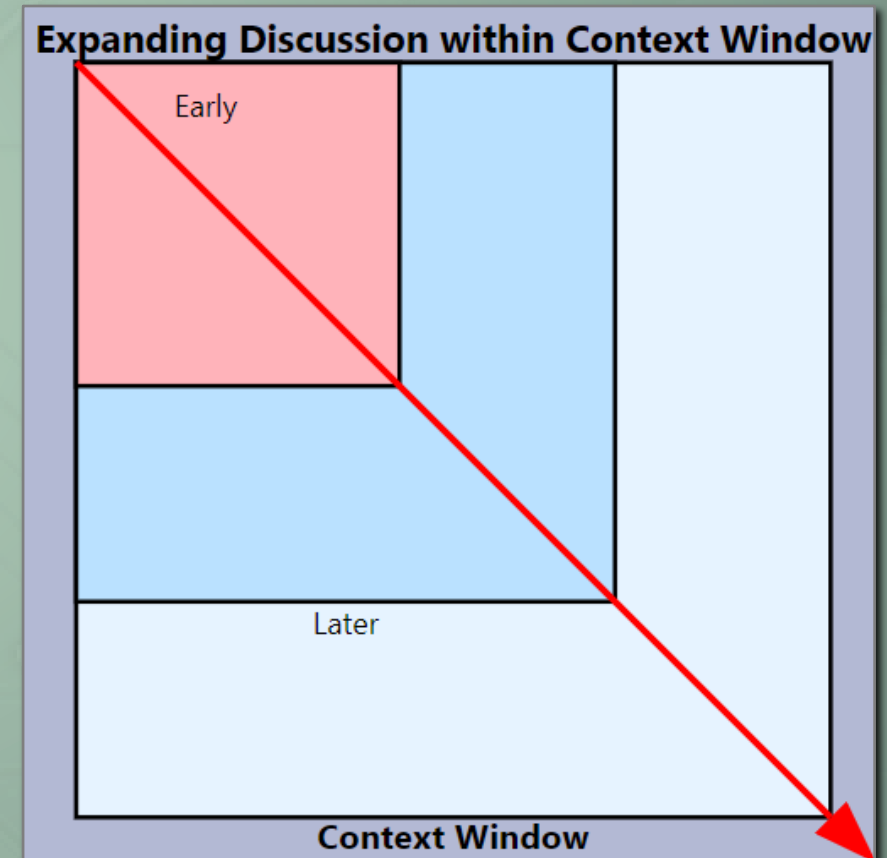


Context window

... or input window, token limit, refers to maximum amount of text (i.e. tokens) that a language model can process and consider at one time when generating responses

- Conversation thread of LLM is basis for referring next questions, i.e. answer to questions also depends on
 - conversation history
 - Additional prompt information, like instruction set, context, system prompt
- All conversation history and question are
 - Tokenized
 - Squared
 - All tokens tuples are evaluated, i.e. attention is calculated
- Based on it, next most probable token is generated.

→ Context window is maximal size of thread



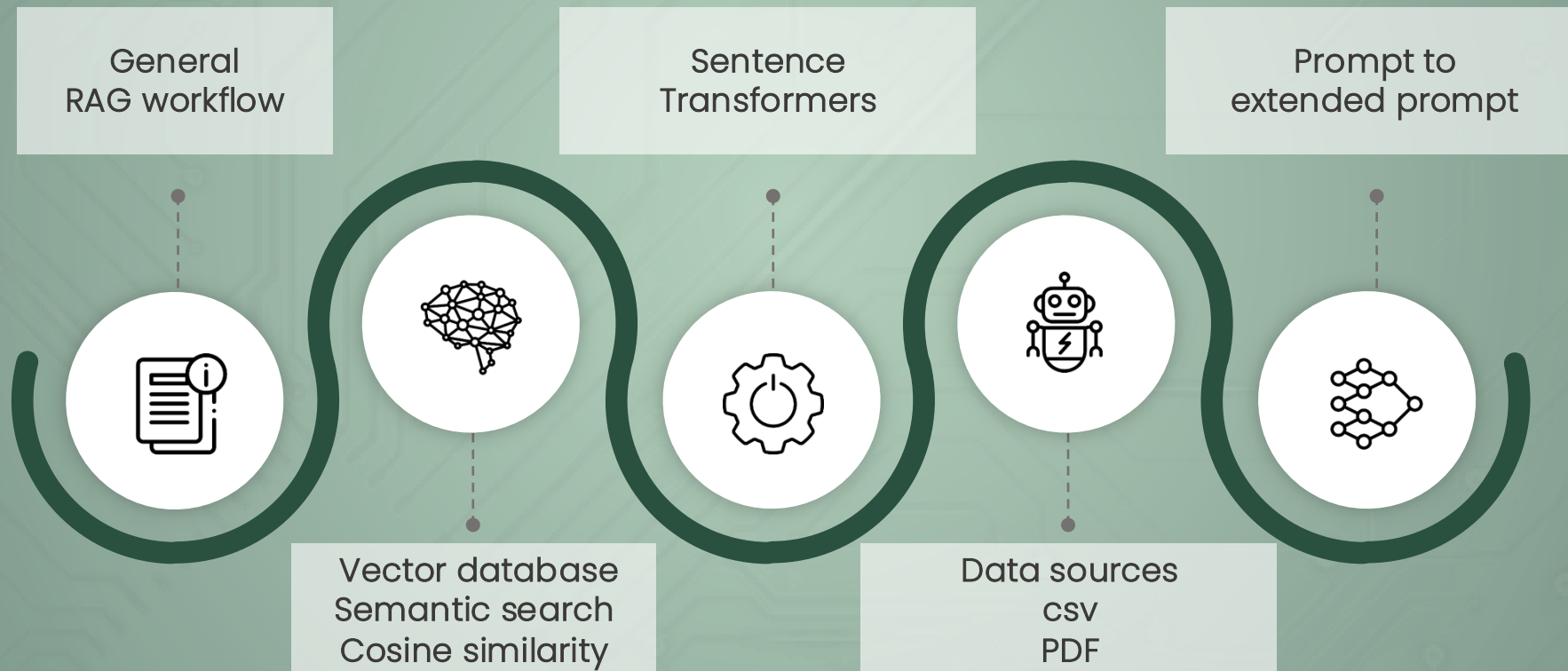
Context window

... or input window, token limit, refers to maximum amount of text (i.e. tokens) that a language model can process and consider at one time when generating responses

LLM	OpenAI GPT-4	Anthropic Claude Sonnet 3.5	Llama 3.1 70B	Mistral 7B	Gemini 1.5
Input Context window (number tokens)	128k	200k	128k	32k	1M (up to 2M)
Maximum output tokens (number tokens)	4096	4096	2048	4096	8192
Words	24.000	48.000	24.000	6.000	192k (up to 384k)
Pages	48	96	48	8	384 (up to 768)

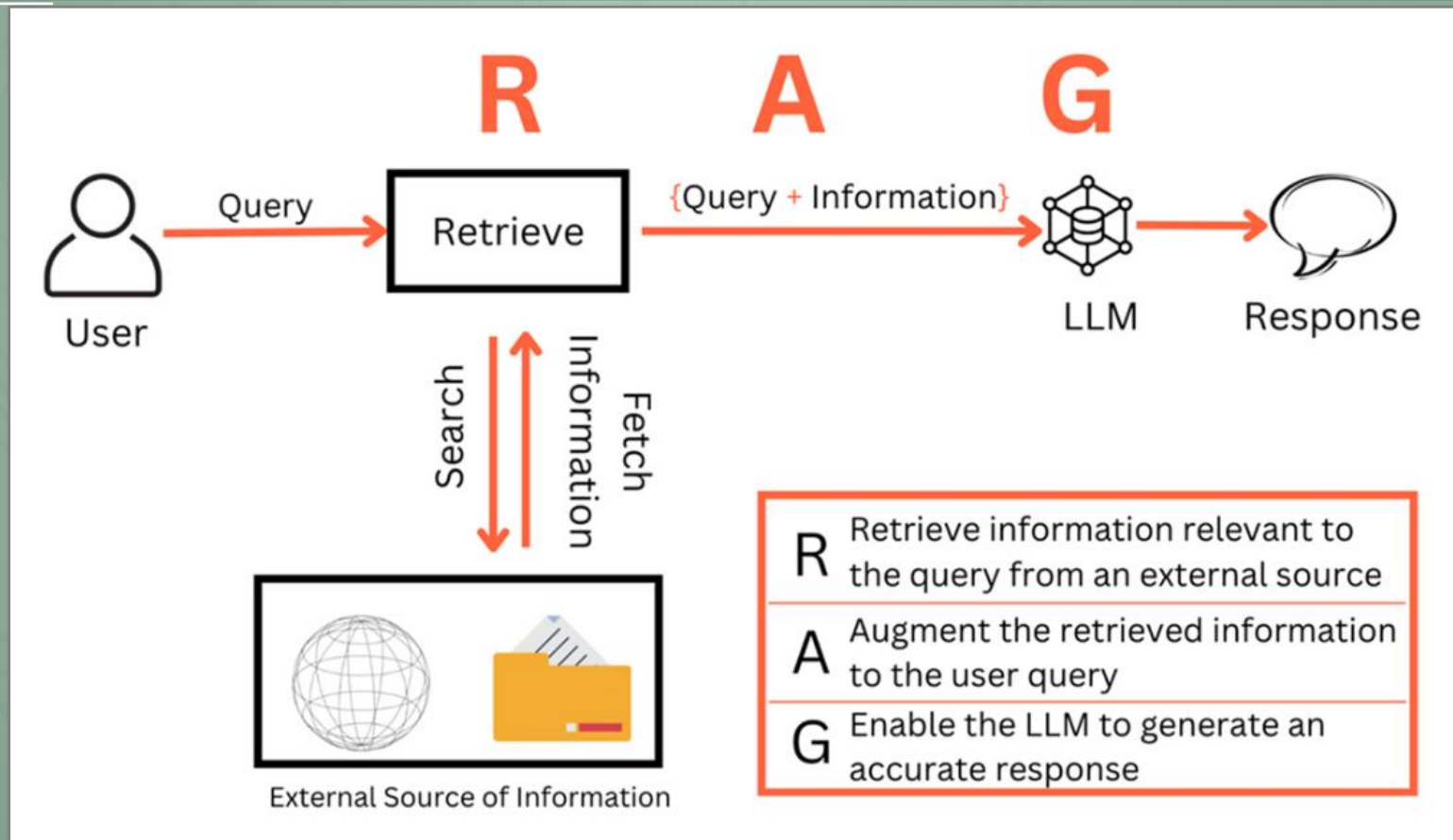
Data Augmentation: Retrieval Augmented Generation

... enhances LLMs by retrieving relevant information from external sources before generating response, allowing model to access up-to-date, specific information beyond training data



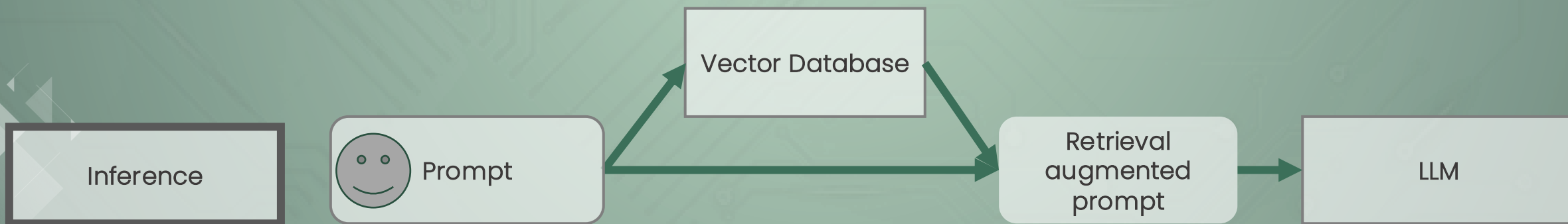
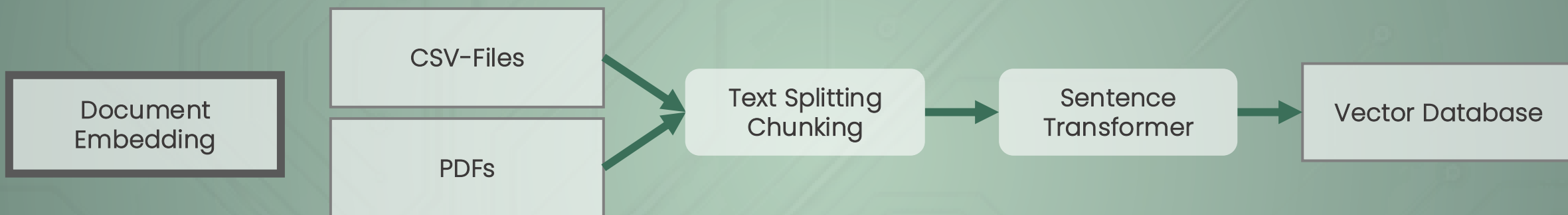
Retrieval Augmented Generation (RAG)

... enhances language model responses by integrating relevant external information with user queries before processing by a language model



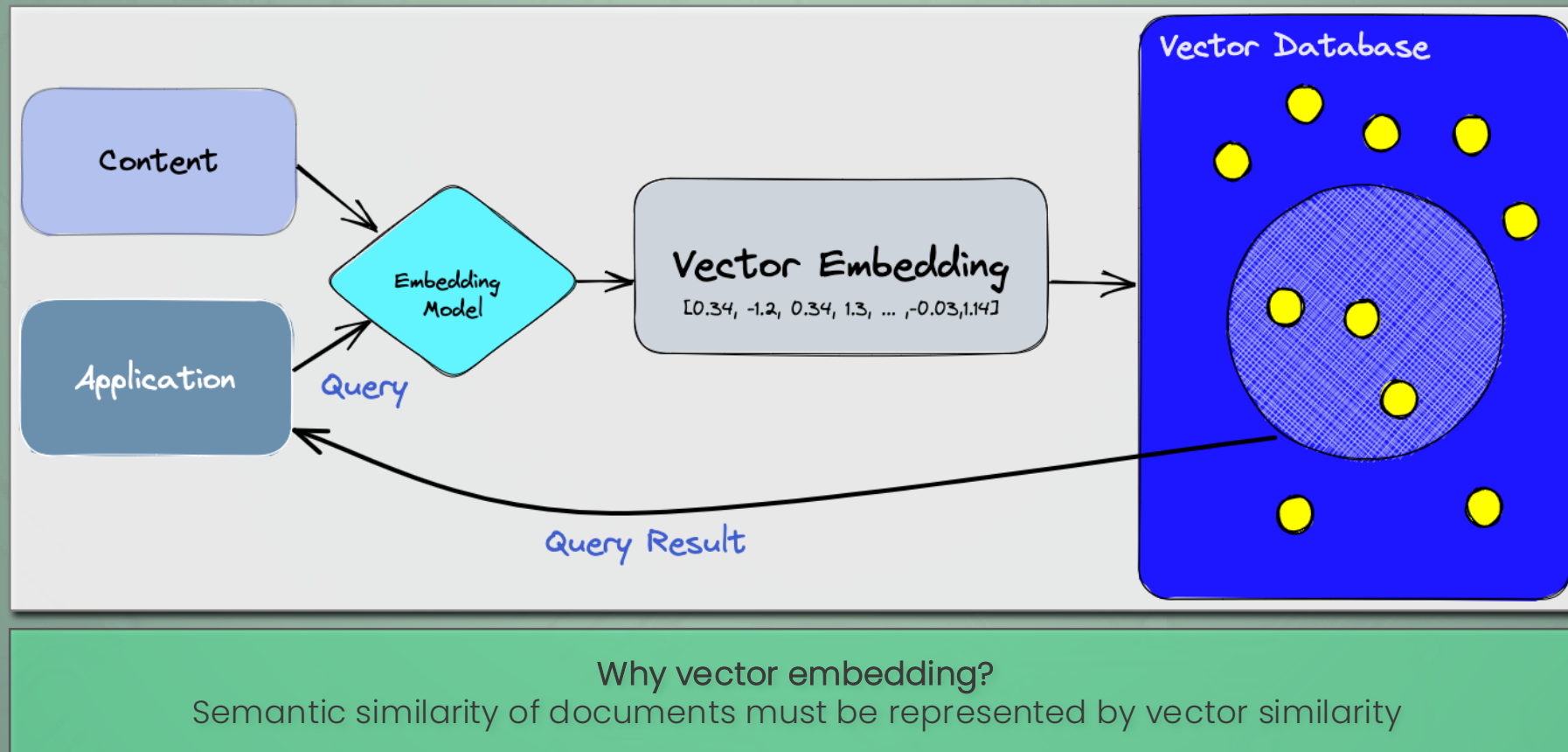
Document embedding vs. inference using vector database

Document embedding transforms data into vectors; inference combines query with relevant vectors for LLM processing.



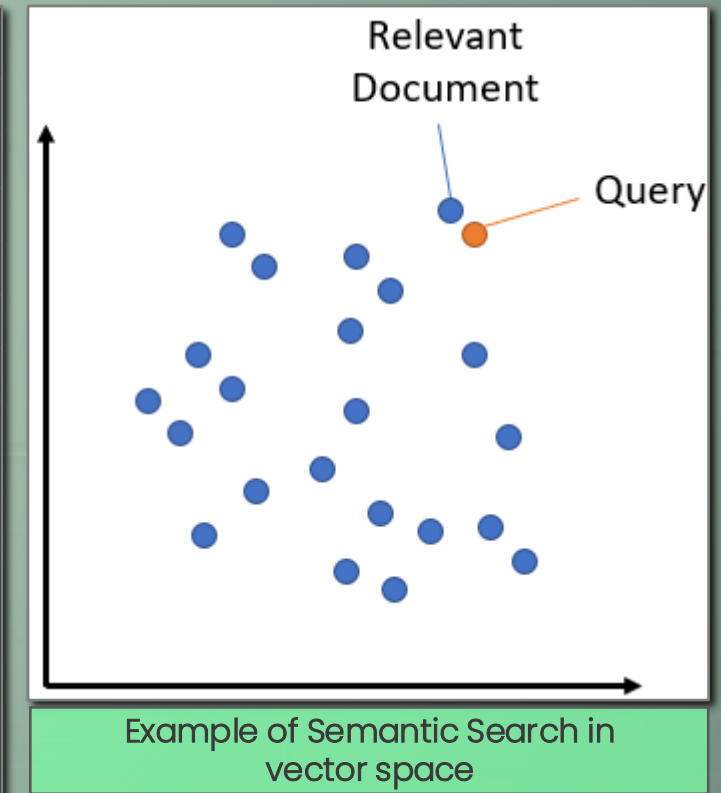
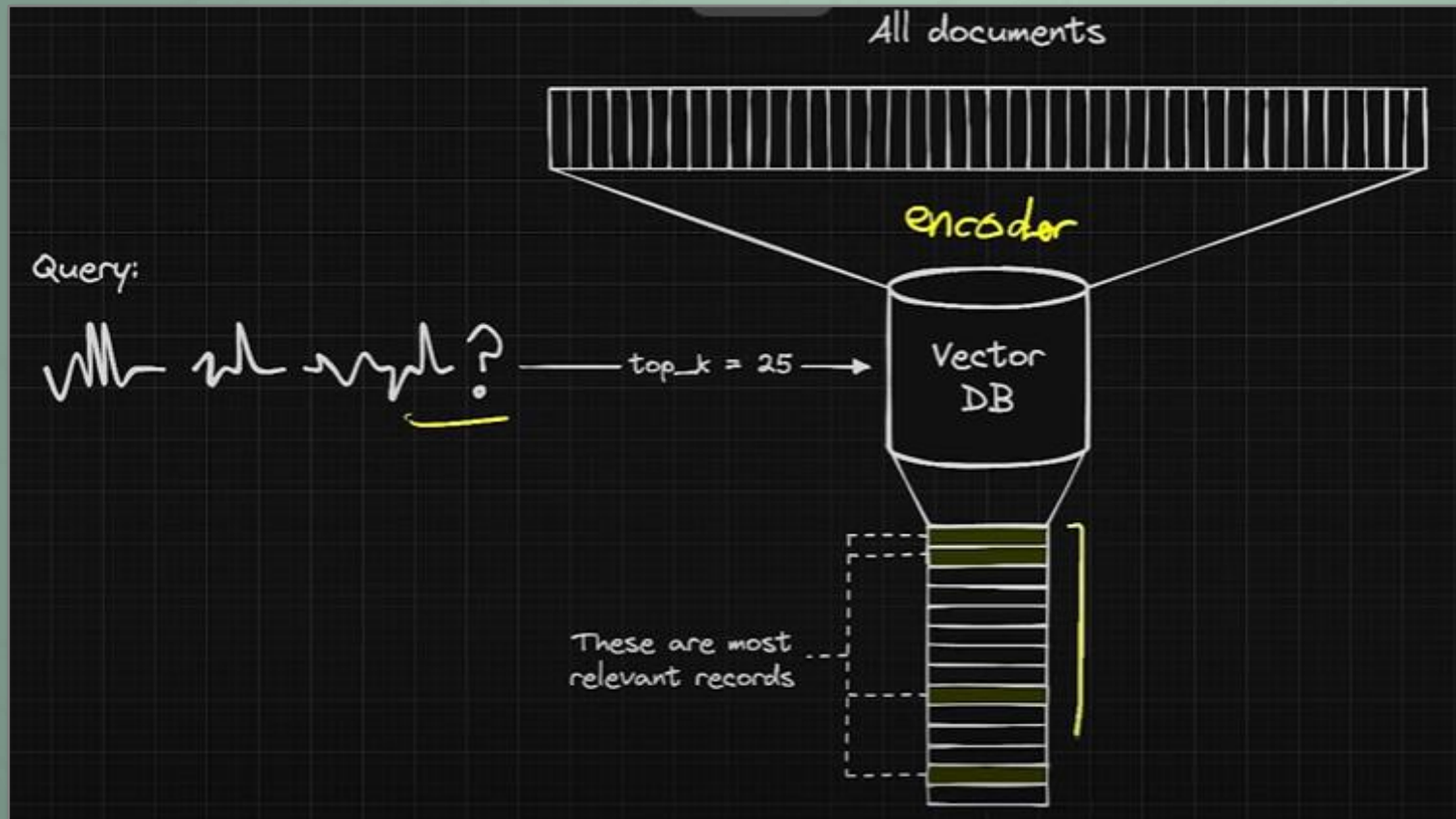
Vector Databases

... are designed to store, manage, query high-dimensional vector data efficiently, being optimized for semantic similarity search operations on large-scale datasets of embeddings.



Vector Databases

Based on query / prompt the vector database provides k (here = 25) most relevant records based on semantic similarity search, e.g. with cosine similarity



Cosine similarity

... is metric used to measure how similar two documents represented by vectors are irrespective of their size. It is commonly used for semantic search in vector databases.

It is measured by the cosine of the angle between two vectors and determines whether two vectors are pointing in roughly the same direction.

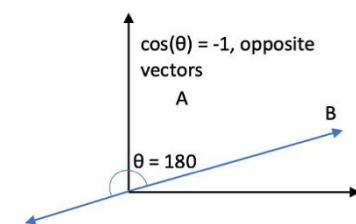
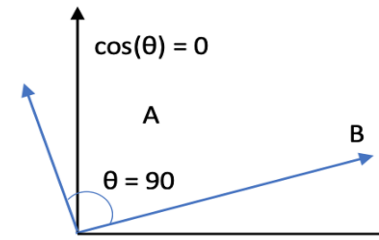
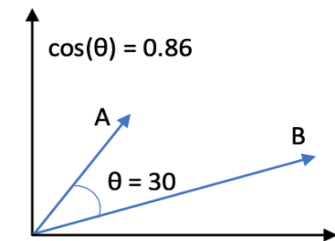
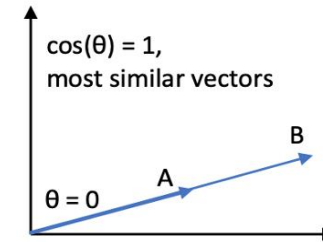
Formula

$$\cos \theta = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \|\vec{b}\|} \text{ with } \|\vec{a}\| = \sqrt{a_1^2 + a_2^2 + \dots + a_n^2} \text{ and } \|\vec{b}\| = \sqrt{b_1^2 + b_2^2 + \dots + b_n^2}$$

- $\vec{a}$ and $\vec{b}$ are vectors, $\vec{a} \cdot \vec{b}$ is their dot product
- $\|\vec{a}\|$ and $\|\vec{b}\|$: magnitudes i.e. lengths of vectors $\vec{a}$ and $\vec{b}$

Explanation

- Ranges from -1 meaning exactly opposite, to 1 meaning exactly the same
- 0 usually indicating independence
- in-between values indicating intermediate similarity or dissimilarity



Documents in vector database

... can be listed by similarity

- Image shows Pinecone vector database management interface.
- Displayed are details of index named "ksb5-lb1twwt ..."
- index uses cosine similarity with 1024 dimensions, contains 2,478 records
- Hosted on AWS, us-west-2 region, serverless
- Query results show matches with
 - IDs
 - vector values
 - similarity scores
 - metadata.
- metadata includes German text descriptions related to VW Tiguan R and its buyers.

The screenshot displays the Pinecone web interface. On the left is a sidebar with navigation options: Chatbot, DATABASE (Indexes (10), Collections), SOLUTIONS (Assistant (Beta)), and MANAGE (API keys, Members). The main panel shows the details for an index named "ksb5-lb1twwtmt3blbkfjdrsgln5v1hp8s8u8ps6w".

Index Details:

- Metric:** cosine
- Dimensions:** 1024
- Host:** https://ksb5-lb1twwtmt3blbkfjdrsgln5v1hp8s8u8ps6w-zanw19t.svc.apw5-4e34-81fa.pinecone.io
- Cloud:** AWS
- Region:** us-west-2
- Type:** Serverless
- Record Count:** 2,478

Query Interface:

Under the "BROWSER" tab, the "Index records" section includes a "Query" button, "List/Fetch", and "Add a record". The query configuration shows:

- Namespace:** (Default)
- Query by:** dense vector value
- vector:** 0.11,0.8,0.24,0.84,0.65,0.26,0.85,0.26,0.76,0.25,0.6,0.82,0.74,0.25,0.43,0.02,0.94,0.14,0.33,0.09,0...
- Top K:** 10

A "Metadata Filter" button is also present.

Matches: 10

The results are displayed in a table with 3 rows shown:

	ID	VALUES	SCORE	METADATA
1	1750	0.0105372136, -0.0310492944, -0.0261904653, -0.0506044067, 0.0616761222, -0.00100327993, 0.0298800077, 0.0718915761, 0.07...	0.0299	text: "VW Tiguan R; Familienstand; Viele Käufer des VW Tiguan R sind verheiratet und haben Kinder. Sie schätzen die Geräumigkeit und die Sicherheitsmerkmale des Fahrzeugs."
2	1560	0.0243917871, -0.0251906607, -0.0289275572, -0.0341392905, 0.0510625616, -0.00871608779, 0.0139721418, 0.0535145961, 0.061...	0.0293	text: "VW Tiguan R; Wohnsituation; Die Käufer des VW Tiguan R leben oft in Eigenheimen oder Mehrfamilienhäusern. Sie schätzen den Komfort und die Sicherheit, die dieses Fahr..."
3	1536	0.0284572076, -0.044948902, -0.0095955329, -0.0327582322, 0.0535162315, -0.0293730572, 0.0223597698, 0.0449360944, 0.0...	0.0292	text: "VW Tiguan R; Interesse Politik; Die Käufer des VW Tiguan R sind oft politisch interessiert und informiert. Sie schätzen die Stabilität und Sicherheit, die ein gut geführtes Lan..."

Typical applications of Vector Databases

... go beyond semantic search which is used for RAG

Focus



Semantic Search

Enable meaning-based search beyond keyword matching



AI and Machine Learning

Power recommendation systems and natural language processing



E-commerce

Improve product recommendations and similar item suggestions



Image and Video Analysis

Enable visual search and content-based image retrieval



Fraud Detection

Identify patterns and anomalies in financial transactions

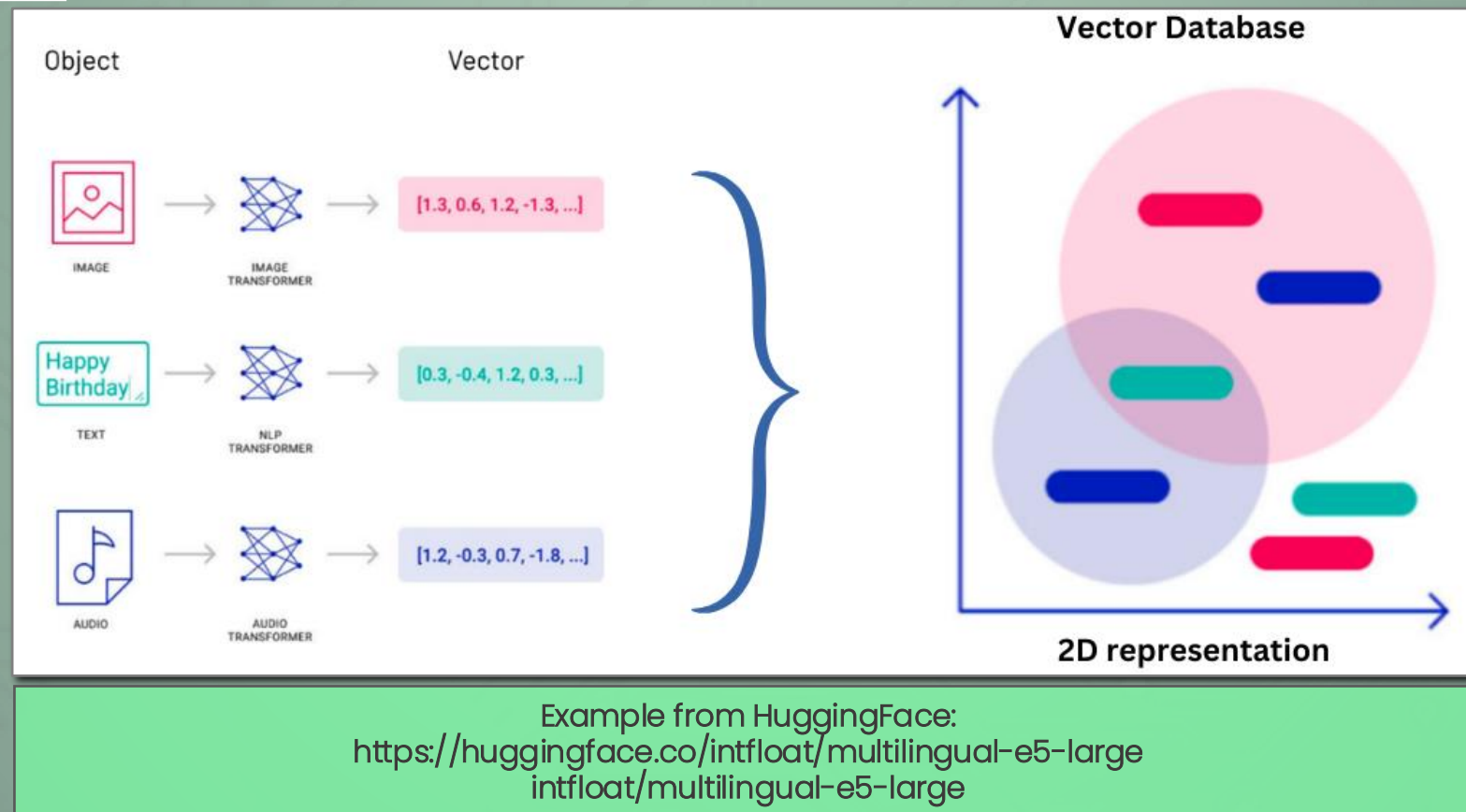


Content Recommendation

Enhance user experience in streaming platforms and social media

Sentence transformers

... are BERT based models designed to convert sentences, paragraphs, longer texts into fixed-size dense vector representations (embeddings) that capture semantic meaning.



Connecting data sources like CSV-, PDF-files

... via Python pipeline to vector database can be tedious, as tables, columns etc. have to be vector embedded in a way that they can be semantically found, ranked etc. afterwards

Ship_name	Cruise_line	Age	Tonnage	passengers	length	cabins	passenger_density	crew
Journey	Azamara	6	30.277	6.94	5.94	3.55	42.64	3.55
Quest	Azamara	6	30.277	6.94	5.94	3.55	42.64	3.55
Celebration	Carnival	26	47.262	14.86	7.22	7.43	31.8	6.7
Conquest	Carnival	11	110	29.74	9.53	14.88	36.99	19.1
Destiny	Carnival	17	101.353	26.42	8.92	13.21	38.36	10
Ecstasy	Carnival	22	70.367	20.52	8.55	10.2	34.29	9.2
Elation	Carnival	15	70.367	20.52	8.55	10.2	34.29	9.2
Fantasy	Carnival	23	70.367	20.56	8.55	10.22	34.23	9.2

Python
pipeline



[1.3, 0.6, 1.2, -1.3, ...]

[0.3, -0.4, 1.2, 0.3, ...]

[1.2, -0.3, 0.7, -1.8, ...]

Meta-Tags:

- Rows
(e.g. ship name)
- Columns
(e.g. feature)

Kid's News

Hammond Elementary School

Difficult
task!

Kids' Right to Vote

by Lindsey

The election is a very important time of the year. Kids should be able to vote. They should be driven by their parents to the polling location to vote and then taken home by their parents. If kids voted it would make a difference in which president was elected. Adults have no more rights than kids, and everybody should have their own right to vote for the president. If kids could vote, the law should be that they can't talk about their votes, and they should have to know a lot about the candidate to vote for him or her. When the next election comes,

information on our political system to be able to make a good decision. Kids don't understand the views of the candidates on issues such as the country's health system, or welfare system, or taxes. They will easily be persuaded by other kids' opinions about whom they are voting for. Kids would be voting without knowing what they are really voting for. They might copy someone else's ballot.

The law should say that if kids are ever allowed to vote, they should be supervised. Kids aren't very educated in politics. If kids knew more about the candidates, it would be fine for them to vote. Although all adults are allowed to vote, some adults vote without

Harry Potter

by Bryan

Harry Potter is a good book for advanced readers. It is a story about a boy who lost his parents and was delivered to his aunt and uncle and his cousin. Since they hate him so much, they make him work hard. One day he gets a letter saying he can go to a wizarding school, but his uncle won't let him go. Then a wizard comes and takes Harry school shopping. Next, Harry is taken to a magic train to a school of witchcraft and wizardry and has a lot of adventures there. One adventure is that he finds out the story of how his parents died.

Python
pipeline



Content needs
to be chunked into
small pieces

[1.3, 0.6, 1.2, -1.3, ...]

[0.3, -0.4, 1.2, 0.3, ...]

Retrieval augmented prompt

Prompt template is filled with retrieved data from VectorDB and passed to LLM

Python example of prompt template

```
from string import Template

migratory_birds_template = Template("""
Migratory Birds (Zugvögel) and Their Winter Habitats
1. ${species1}: Summers in ${summer1}, winters in ${winter1}
2. ${species2}: Summers in ${summer2}, winters in ${winter2}
3. ${species3}: Summers in ${summer3}, winters in ${winter3}
Interesting fact: ${fact}

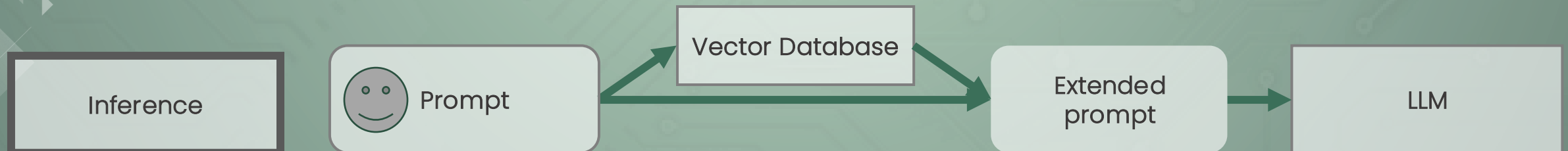
Conservation of these habitats is crucial for these birds'
survival.
""")
```

Retrieved information from vector database, sorted by relevance score

```
retrieved_info = [
    {
        'species': 'Arctic Tern',
        'summer_habitat': 'Arctic regions',
        'winter_habitat': 'Antarctic coast',
        'fact': 'Makes the longest known migration of any animal,
flying about 70,900 km (44,100 miles) round trip annually.',
        'relevance_score': 0.95
    },
    {
        'species': 'Bar-tailed Godwit',
        'summer_habitat': 'Alaska',
        'winter_habitat': 'New Zealand',
        'fact': 'Can fly over 11,000 km non-stop, the longest known
non-stop flight of any bird.',
        'relevance_score': 0.92
    },
    (...)
]

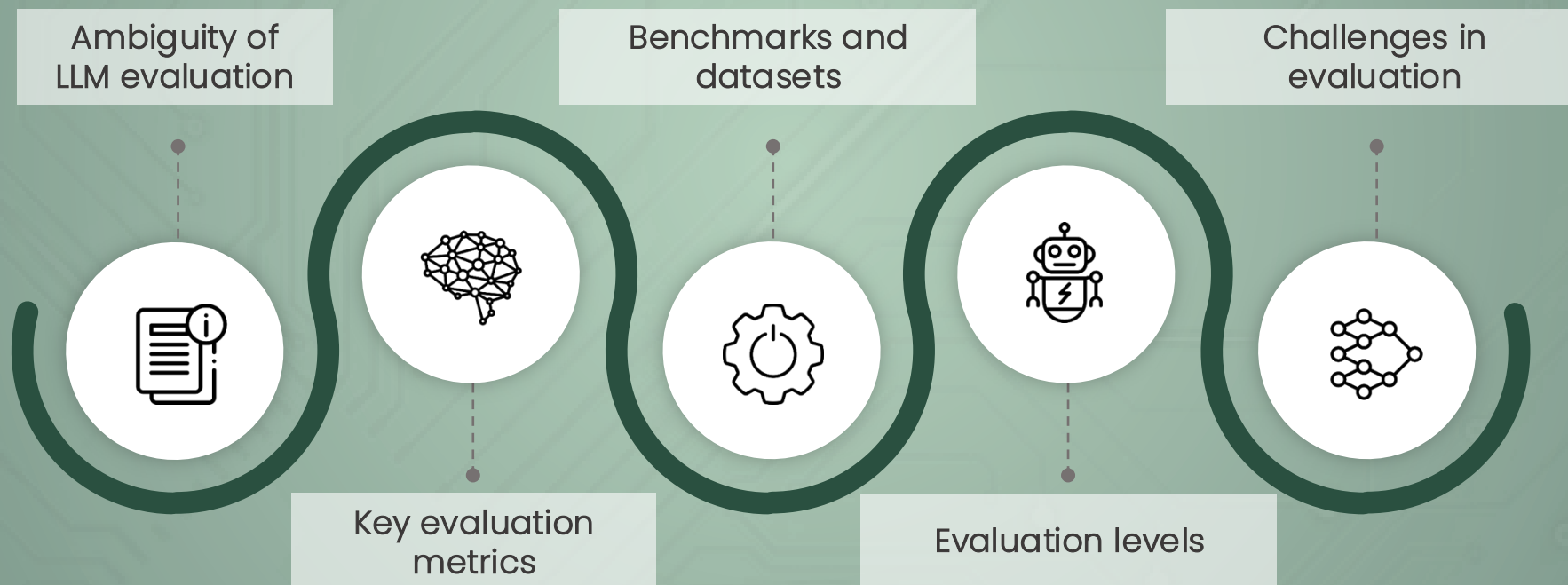
bird_info = {
    'species1': retrieved_info[0]['species'],
    'summer1': retrieved_info[0]['summer_habitat'],
    'winter1': retrieved_info[0]['winter_habitat'],
    (...)
}

formatted_prompt = migratory_birds_template.safe_substitute(bird_info)
print(formatted_prompt)
```



Evaluating Large Language Models

... involves assessing performance across diverse tasks, like natural language understanding, generation, reasoning, while considering accuracy, fluency, coherence, bias, ethics



What do AI agents and driving have in common?

For both, the question arises: how many per mille is still okay?
(blood alcohol content vs. LLM generation error)

Language Models inherently make mistakes due to

- Lack of necessary domain data
- **Hallucinations:**
Even strong models with proper prompts may hallucinate on per mille scale

Therefore quality must be measurable

Problem

- Evaluation is often ambiguous because quality measure depends on given task
- Hence, quality measures can only refer to particular task



Key evaluation metrics

Precision: How many generated words/phrases are correct or relevant?

Recall: How many correct or relevant elements are captured?

Metric	Description	Use Case	Pros	Cons
Perplexity	- measures how well probability model predicts sample - indicating model's uncertainty in its predictions - The lower the better	- Text generation - Speech recognition	- Good for comparing models - Intrinsic measure of model quality, does not require human in the loop	- Doesn't capture semantic meaning or coherence of generated text - only the model's confidence in its predictions
BLEU Score (Bilingual Evaluation Understudy)	- Algorithm for evaluating quality of machine-translated text - compares n-grams of candidate translation with n-grams of reference translation(s) - range from 0 to 1, where 1 indicates perfect translation	Machine translation	- Widely used in translation tasks - Correlates with human judgements	- BLEU rewards translations that are accurate with the words they use, but may not capture all information from the source (Precision oriented) - Doesn't account for meaning or fluency
ROUGE Score (Recall-Oriented Understudy for Gisting Evaluation)	- Set of metrics for evaluating automatic summarization and machine translation - Measures overlap between generated and reference texts	- Text summarization - Text generation	- Multiple variants for different aspects - Considers both precision and recall	- Relies on exact word matches - May not capture paraphrasing well
F1 Score	Harmonic mean of precision and recall	- Classification tasks - information retrieval	- Balances precision and recall - Good for imbalanced datasets	- May not be suitable for all NLP tasks, focus on classification - Doesn't account for true negatives

Benchmarks and datasets

... are standardized tools for evaluating and comparing LLMs' performance across diverse language tasks, from basic comprehension to complex reasoning.

Benchmark, Dataset	Description	Tasks	Key Features	Challenges
GLUE	General Language Understanding Evaluation benchmark	▪ Sentence similarity ▪ Natural language inference ▪ Sentiment analysis	▪ Diverse set of tasks ▪ Well-established in NLP community	▪ Some tasks considered too easy for modern LLMs ▪ Limited to English language
SuperGLUE	More challenging successor to GLUE	▪ Question answering ▪ Coreference resolution ▪ Causal reasoning	▪ More difficult tasks than GLUE ▪ Closer to human-level performance benchmarks	▪ Still primarily focused on English ▪ May not fully capture capabilities of most advanced LLMs
SQuAD	Stanford Question Answering Dataset	▪ Reading comprehension ▪ Question answering	▪ Large-scale dataset ▪ Real-world questions and contexts	▪ Answers limited to spans within given text ▪ May not test deeper reasoning capabilities
LAMBADA	Language Model Benchmark on cloze-style task	▪ Word prediction ▪ Language modeling	▪ Tests broad context understanding ▪ Requires common sense reasoning	▪ Focus on specific last-word prediction task ▪ May not cover full range of language understanding

Evaluation levels

... are ranging from zero-shot, few-shot to fine-tuning, balancing generalization and task-specific capabilities.

Approach	Description	Advantages	Limitations	Use Cases
Zero-shot	Model performs task without any task-specific examples	▪ No additional training required ▪ Tests model's general knowledge ▪ Quick to implement	▪ May not perform well on specialized tasks ▪ Highly dependent on prompt engineering	▪ General language understanding tasks ▪ Quick prototyping ▪ Assessing model's base capabilities
Few-shot	Model is given a few examples of the task before performing it	▪ Better performance than zero-shot ▪ No need for model retraining ▪ Flexible for different tasks	▪ Performance can vary based on example selection ▪ Limited by context window size	▪ Task adaptation with limited data ▪ Evaluating model's ability to learn from examples
Fine-tuning	Model is retrained on task-specific data	▪ Best performance on specific tasks ▪ Can learn task-specific knowledge ▪ Adaptable to specialized domains	▪ Requires significant computational resources ▪ Needs substantial task-specific data ▪ Risk of overfitting to specific task	▪ Specialized applications ▪ When high performance on specific tasks is crucial ▪ Domains with ample task-specific data

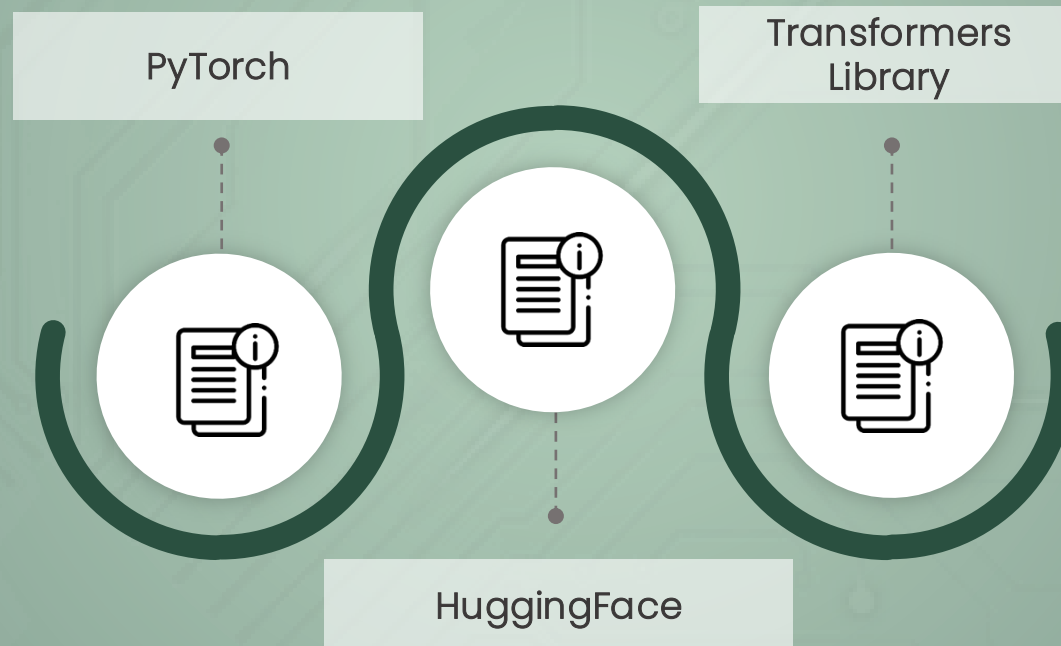
Challenges in evaluating Language Models

... range from addressing bias, contextual understanding to ensuring scalability, ethical compliance, requiring sophisticated and responsible assessment methods.

Challenge Area	Key Issues
Bias and Fairness	▪ Detecting and quantifying inherent biases ▪ Ensuring fairness across diverse groups ▪ Balancing performance with ethical considerations
Contextual Understanding	▪ Evaluating nuanced language comprehension ▪ Assessing performance in domain-specific contexts ▪ Measuring ability to handle ambiguity
Robustness and Generalization	▪ Testing performance on out-of-distribution data ▪ Evaluating consistency across various inputs ▪ Assessing adaptability to new tasks or domains
Scalability of Evaluation	▪ Managing computational resources for large-scale testing ▪ Developing efficient evaluation pipelines ▪ Balancing thoroughness with practicality
Ethical and Safety Considerations	▪ Assessing potential for misuse or harmful outputs ▪ Evaluating alignment with human values ▪ Measuring truthfulness and factual accuracy ▪ Ensuring privacy and data protection in evaluation

Programming Tools

...involves the organization, summarization, and visualization of data. It provides simple summaries about the sample and the measures.



What is PyTorch?

It is an open-source deep learning framework developed by Facebook's AI Research lab.

Key Features:

- Dynamic computational graph, offering flexibility in model design.
- Native support for GPU acceleration, enhancing performance.
- Intuitive tensor library similar to NumPy but with GPU support.

Applications:

- Research prototyping, offering ease and speed.
- Building deep learning models, from neural networks to complex architectures.

Community & Ecosystem:

- Active community contributing to its growth.
- Rich ecosystem with pre-trained models, tools, and libraries.



TensorFlow, PyTorch and Keras

... are popular open-source libraries used for machine learning and deep learning.
Keras is API built on top of TensorFlow, PyTorch and others

	1  TensorFlow	2  PyTorch Focus	3  Keras
DEVELOPED BY	Google Brain team	Facebook's AI Research lab	François Chollet
EASE OF USE	Steeper learning curve	Easier to learn	Extremely user-friendly
FLEXIBILITY / CUSTOMIZATION	Very flexible	Very flexible	Limited flexibility
PERFORMANCE / SPEED	Excellent performance	Excellent performance	Performance is dependent on the backend used
USE CASES	Suitable for both production and research, especially in large-scale systems	Preferred for research, rapid prototyping, and academic purposes	Best for quick development of standard neural networks, less suited for research

HuggingFace

... offers 100s of 1000s NPL models, datasets, tokenizers, etc. and has vibrant open source community of AI researchers, engineers, enthusiasts contributing to AI advancement

Focus

Transformers

🔗 119,930

State-of-the-art ML for Pytorch, TensorFlow, and JAX.

Tokenizers

🔗 8,111

Fast tokenizers, optimized for both research and production.

Diffusers

🔗 20,869

State-of-the-art diffusion models for image and audio generation in PyTorch.

Safetensors

🔗 2,022

Simple, safe way to store and distribute neural networks weights safely and quickly.

Hub Python Library

🔗 1,492

Client library for the HF Hub: manage repositories from your Python runtime.

Transformers.js

🔗 5,779

Community library to run pretrained models from Transformers in your browser.

PEFT

🔗 12,469

Parameter efficient finetuning methods for large models

timm

🔗 28,648

State-of-the-art computer vision models, layers, optimizers, training/evaluation, and utilities.

HuggingFace: Transformers Library

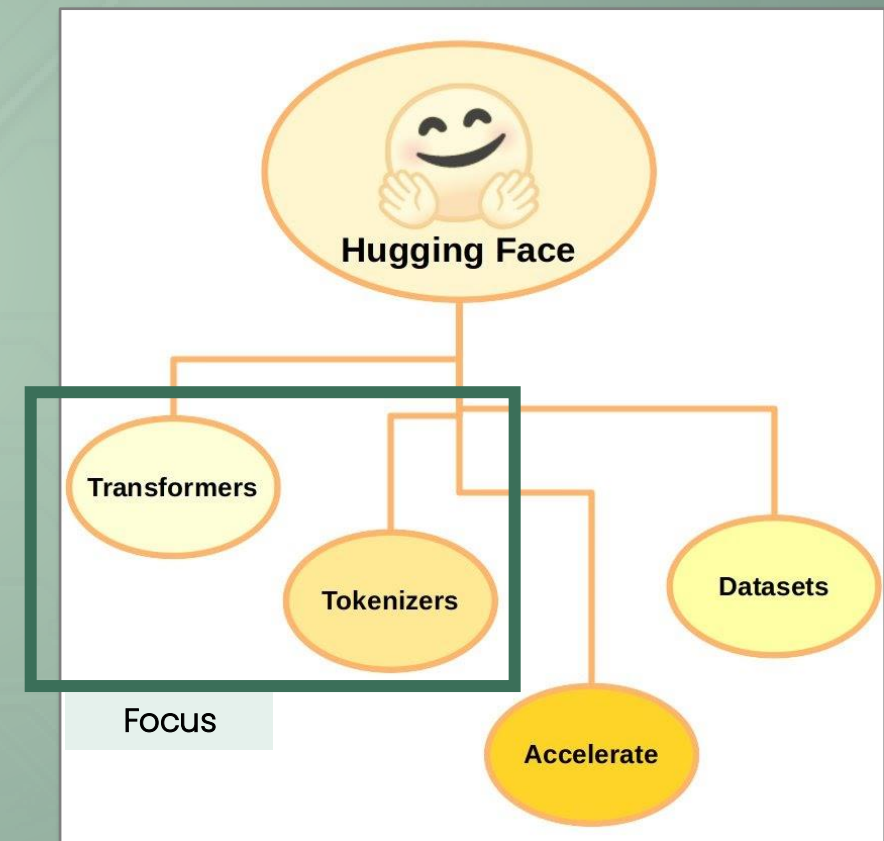
... provides tools to easily download, train state-of-the-art pretrained models. Using pretrained models can reduce time and resources required to train a model from scratch

Key Features:

- **Comprehensive and Modular:**
Offers models like BERT, GPT, T5, etc., with simple interface for NLP tasks.
- **Ease of Use:**
Enables easy model downloading, training, deployment with minimal code.
- **Community-Driven:**
Actively supported by vast community, ensuring continuous updates, enhancements.

Applications:

- Research prototyping, offering ease and speed.
- Building deep learning models, from neural networks to complex architectures.



Links

i.e. sources for self-learning

	Title	Link
Basics of Natural Language Processing	An Introduction to Bag of Words (BoW) What is Bag of Words?	An Introduction to Bag of Words (BoW) What is Bag of Words?
	Understanding TF-IDF: A Traditional Approach to Feature Extraction in NLP	https://towardsdatascience.com/understanding-tf-idf-a-traditional-approach-to-feature-extraction-in-nlp-a5bfbe04723f
	Word Embedding and Word2Vec, Clearly Explained!!!	https://www.youtube.com/watch?v=viZrOnJcIY0
	Word2Vec Explained	https://towardsdatascience.com/word2vec-explained-49c52b4ccb71
	Word2vec : NLP & Word Embedding	https://datascientest.com/de/word2vec
	Stanford XCS224U: NLU I Intro & Evolution of Natural Language Understanding	https://www.youtube.com/watch?v=K_Dh0Sxujuc&list=PLoROMvodv4rOwvldxftJTmoR3kRcWkJBp
	Introduction NLP Tutorial For Beginners In Python	https://www.youtube.com/watch?v=R-AG4-qZs1A&list=PLeo1K3hjS3uuvuAXhYjV2IMEShq2UYSwX

Links

i.e. sources for self-learning

	Title	Link
Transformer based NLP	How GPT Works by Archerman Capital	https://www.youtube.com/watch?v=Mhy7I4E6eXs
	The math behind Attention: Keys, Queries, and Values matrices	https://www.youtube.com/watch?v=UPtG_38Oq8o
	The Attention Mechanism in Large Language Models	https://www.youtube.com/watch?v=OxCpWwDCDFQ
	Let's build GPT: from scratch, in code, spelled out.	https://www.youtube.com/watch?v=kCc8FmEbInY
	State of GPT BRK216HFS	https://www.youtube.com/watch?v=bZQun8Y4L2A
	Stanford CS224N: Natural Language Processing with Deep Learning	https://www.youtube.com/playlist?list=PLoROMvodv4rMFqRtEuo6SGjY4XbRIVRd4
	Data Science: Transformers for Natural Language Processing	https://www.udemy.com/course/data-science-transformers-nlp/learn/lecture/32714220
	Awesome-LLM-KG	https://github.com/RManLuo/Awesome-LLM-KG
	Transformers in Action: Attention Is All You Need	https://towardsdatascience.com/transformers-in-action-attention-is-all-you-need-ac10338a023a
	Attention Is All You Need	https://arxiv.org/abs/1706.03762
	BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding	https://arxiv.org/abs/1810.04805
	What exactly happens when we fine-tune BERT?	https://towardsdatascience.com/what-exactly-happens-when-we-fine-tune-bert-f5dc32885d76
	Sentiment Analysis in 10 Minutes with BERT and TensorFlow	https://towardsdatascience.com/sentiment-analysis-in-10-minutes-with-bert-and-hugging-face-294e8a04b671



About me

Dr. Harald Stein

- Data Scientist ~ 9 years experience
- AlgoTrader ~ 4 years experience
- Ph.D. in Economics, Game Theory
- LinkedIn: <https://www.linkedin.com/in/harald-stein-phd-1648b51a>
- ResearchGate: <https://www.researchgate.net/profile/Harald-Stein>

